

WEB TECHNOLOGIES

INTRODUCTION TO WEB TECHNOLOGIES

Client/Server model

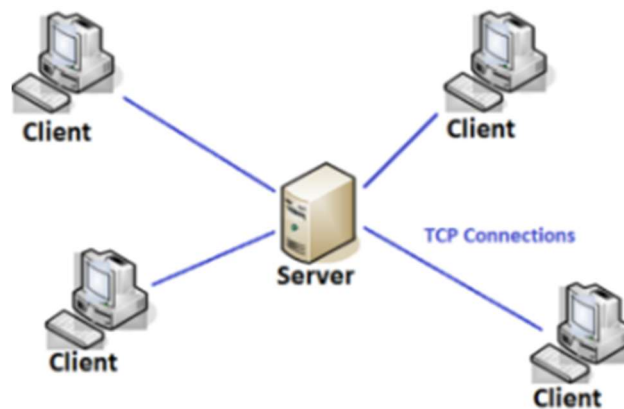
⇒ A model where tasks are divided between clients and servers.

➤ **Client:**

- Requests services or resources.
- Examples: Web browsers, mobile apps, email clients.

➤ **Server:**

- Provides services or resources.
- Examples: Web server, database server, file server.



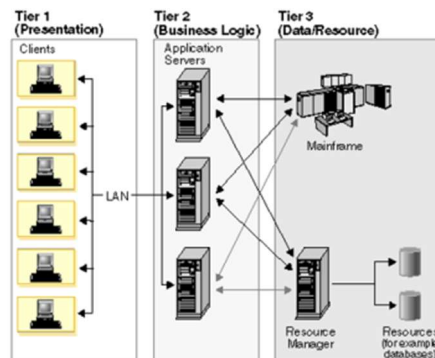
Three-tier Client Server Architecture

➤ **Two-Tier Architecture:**

- **Client Level:** User interface and request handling
- **Server Level:** Processes requests and manages data

➤ **Three-Tier Architecture:**

- **Client:** Interacts with the user
- **Application Server:** Contains business logic
- **Resource Manager:** Stores and manages data



Types of Servers



Web server



Mail server



Application server



Database server



DNS server



Proxy server



DHCP server



File server



Gaming server

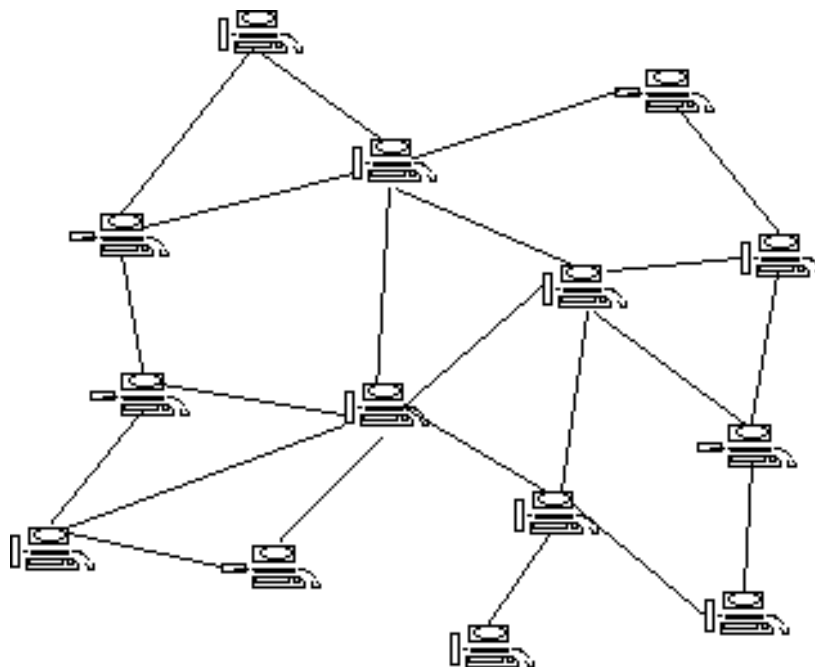


Print server

Peer to Peer (P2P)

⇒ Distributed architecture that divides tasks among peers.

- **Peers:** Equally privileged and capable participants.
- **Network:** Forms a peer-to-peer (P2P) network of nodes.



Internet

- Global network of interconnected devices
- Uses **TCP/IP** protocols for communication

Types of Networks (by Size & Coverage):

- **PAN (Personal Area Network):**
 - Short-range (≤ 10 meters)
 - Example: Bluetooth between phone and headset
- **LAN (Local Area Network):**
 - Covers small areas (home, office, building)
- **MAN (Metropolitan Area Network):**
 - Covers a city or large campus
- **WAN (Wide Area Network):**
 - Covers large areas (country/world)

Network Address Translation (NAT)

⇒ Modifies IP address info in packet headers via a router or firewall.

✓ **Purpose:**

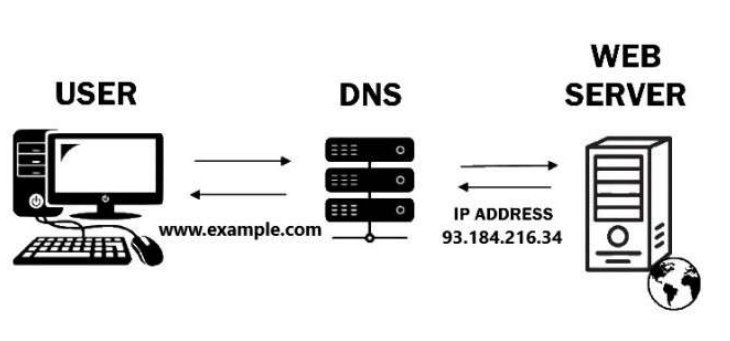
- Share a **single public IP** among multiple devices
- **Conserves IP addresses**
- Enhances **network security**

✓ **IP Address Types:**

- **Private IP:**
 - Used within local networks
 - Not routable on the internet
 - Examples: 192.168.x.x, 10.x.x.x, 172.16.x.x
- **Public IP:**
 - Assigned by ISPs
 - Used for internet communication

Domain Name System

- **Function:** Translates domain names into IP addresses
- **Purpose:** Allows access to websites using names (e.g., www.google.com) instead of IPs
- **Analogy:** Acts like the **phone book** of the internet, mapping names to addresses



World Wide Web (WWW)

⇒ System of interlinked hypertext documents and multimedia accessed via the Internet

- **Inventor:** Tim Berners-Lee (1989)
- **Importance:** Most widely used service on the Internet

Decade	Key Developments
1960s	Development of ARPANET
1980s	Proposal of the World Wide Web by Tim Berners-Lee
1990s	Launch of the first web browser and server
2000s	Dot-com boom, emergence of social media
2010s	Rise of mobile technology, advent of IoT
2020s	Continued innovation and evolution

WWW Standards

- **W3C (World Wide Web Consortium):**
 - Develops standards like **HTML**, **XHTML**, **DOM**
 - Defines structure & interpretation of web documents
- **IETF & Others:**
 - Contribute to internet standards
- **ECMAScript:**
 - Standard for **JavaScript**, by Ecma International
- **DOM (Document Object Model):**
 - From **W3C**, defines how web documents are structured and accessed
- **HTTP (Hyper Text Transfer Protocol):**
 - Protocol for communication & authentication between browser and server
- **URI (Uniform Resource Identifier):**
 - Identifies resources (e.g., documents, images) on the Internet

Uniform Resource Identifier (Example)

http://en.wikipedia.org:80/wiki/URI?page=2&frame=1#Examples_of_URI_references

- **Scheme:** `http` — protocol used
- **Authority:** `en.wikipedia.org` — domain name or server
- **Port:** `80` — communication port
- **Path:** `/wiki/URI` — specific resource location
- **Query:** `page=2&frame=1` — parameters for resource
- **Fragment:** `#Examples_of_URI_references` — section anchor within the page



URI, URL, & URN

- **URI (Uniform Resource Identifier):**
 - Identifies a resource on the Internet
 - Broad term covering URLs and URNs
 - Example: <https://www.example.com/path/to/resource?query=value#section>
- **URL (Uniform Resource Locator):**
 - Specifies resource location and access protocol
 - Example: <https://www.example.com/index.html>
- **URN (Uniform Resource Name):**
 - Provides a unique, persistent name without location or access info
 - Example: `urn:isbn:0451450523`

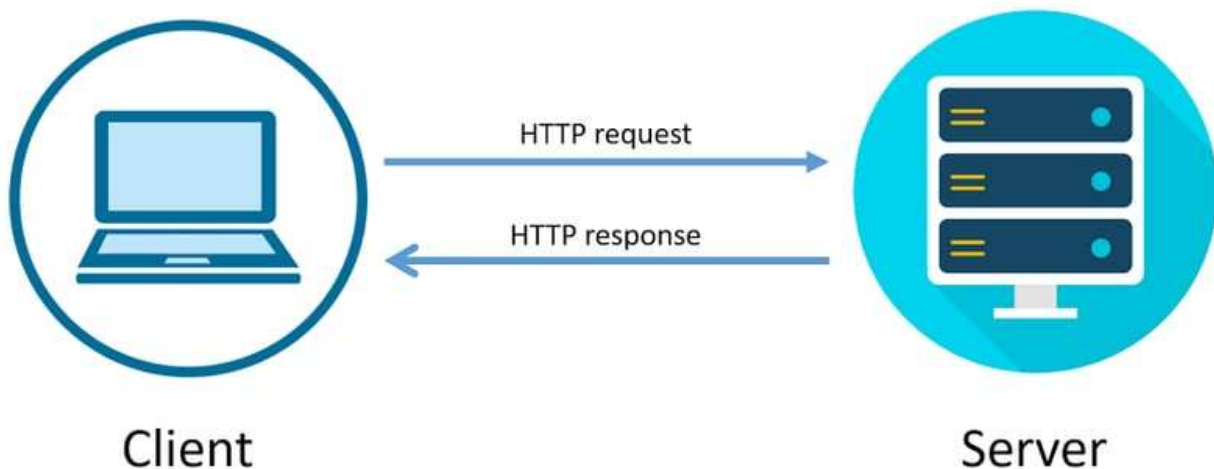
Browsers

⇒ **Definition:** Software application to retrieve, display, and navigate web content

- **Content:** Accesses resources identified by a URI (web pages, images, videos, etc.)
- **Examples:** Internet Explorer, Netscape, Firefox, Opera, Chrome, Safari

HTTP protocol

- **Client:** Sends an **HTTP request** to the server
- **Server:** Returns an **HTTP response** to the client



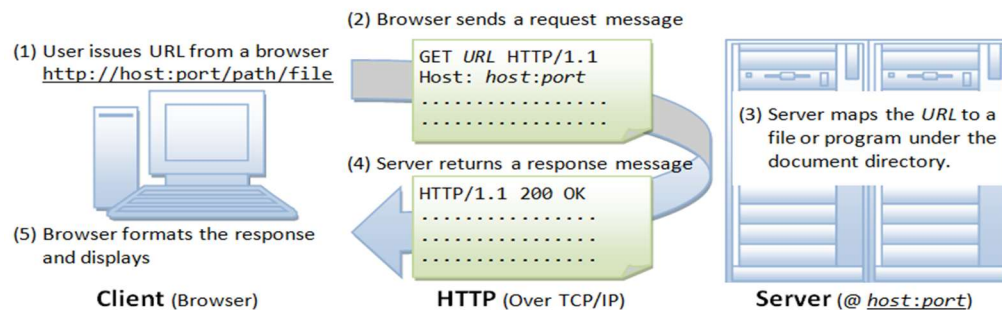
HTTP (HyperText Transfer Protocol):

- **Default Port:** 80
- **Communication:** Via HTTP Requests and Responses
- **Connectionless:** No memory of previous connections after closing
- **Stateless:** Client and server do not retain session information

HTTP Steps

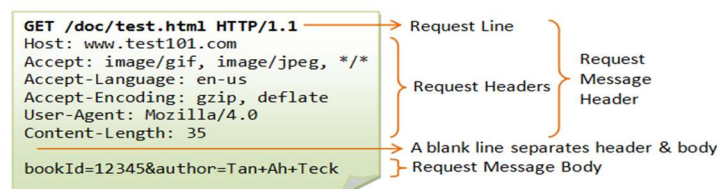
URL to HTTP Request:

- When a **URL** (e.g., <http://www.abc.com/index.html>) is entered in a browser:
 - The browser **converts it into an HTTP request**
 - Sends the request to the **HTTP server** to fetch the resource



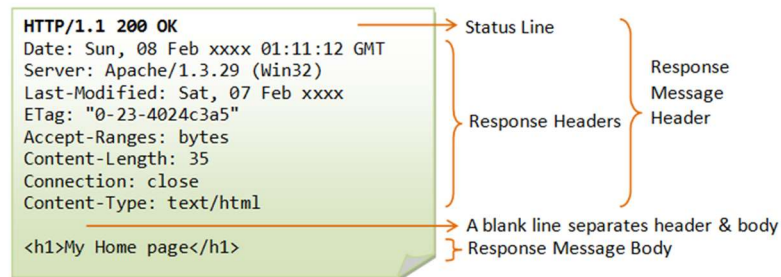
HTTP Server & URL:

- **HTTP Server:**
 - Interprets the request
 - Sends a **response** (requested resource or error)
- **URL (Uniform Resource Locator):**
 - Uniquely identifies a web resource
 - Syntax: protocol://hostname:port/path-and-file-name
- **Browser Action:**
 - Converts URL into a **request message**
 - Sends it to the server using the specified **protocol**



How Server Handles HTTP Requests:

- **Request Mapping:**
 - Maps the request to a **file** in the server's document directory
 - Or maps to a **program**, executes it, and returns the output
- **Response:**
 - Returns the requested **file** or **program output**
 - If not successful, returns an **error message**



HTTP Response Message

Response Types:

- **1xx:** Informational
- **2xx:** Success
- **3xx:** Redirection
- **4xx:** Client Error
- **5xx:** Server Error

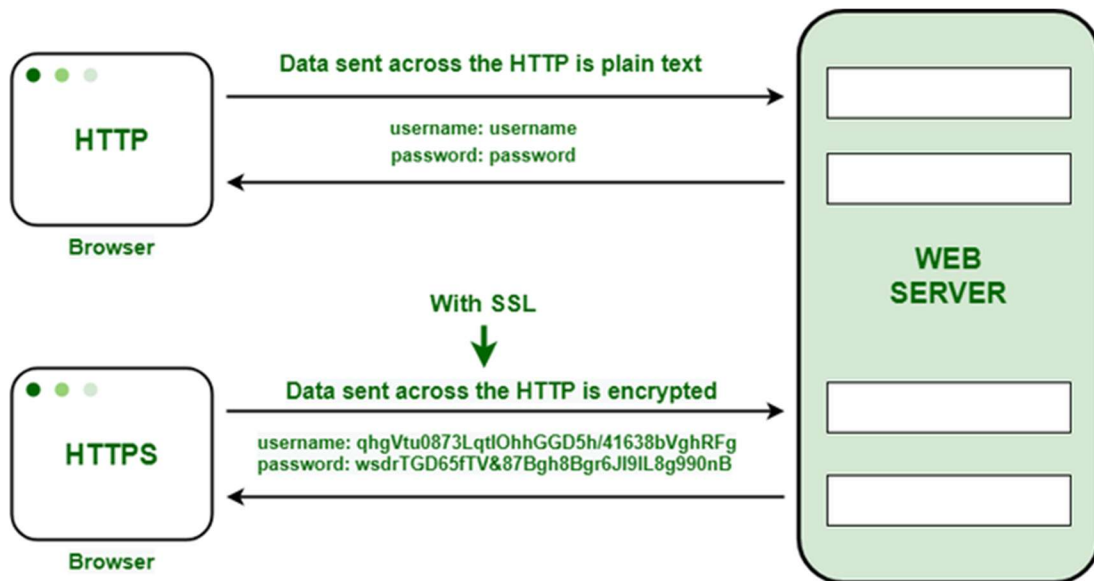
Common Status Codes:

Status Code	Status Text	Meaning
1xx	Informational	Request received, continuing process
200	OK	Request successful; data returned
201	Created	Request successful; new resource created
400	Bad Request	Invalid syntax in request
401	Unauthorized	Authentication required
403	Forbidden	Server understood request but refuses to authorize
404	Not Found	Requested resource not found
500	Internal Server Error	Server encountered an unexpected error

HTTP Request Methods

Method	Purpose
GET	Retrieves a resource from the server
POST	Sends data to the server (e.g., via HTML forms)
PUT	Asks the server to store or update data
DELETE	Requests the server to delete specified data

HTTP Vs HTTPS



HTML

What is HTML?

HTML (HyperText Markup Language):

- Describes the **structure** of a web page
- Made up of **elements** represented by **tags**
- Tags define content types like **heading, paragraph, table**, etc.
- **Browsers use tags** to render content but do not display the tags themselves
- **Invented by Tim Berners-Lee in 1993**

A Simple HTML Document

Example:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    <h1>My First Heading</h1>
    <p>My first paragraph.</p>
  </body>
</html>
```

Explanation:

HTML Elements Explanation:

- **<!DOCTYPE html>:** Declares the document as **HTML5**
- **<html>:** Root element of the HTML page
- **<head>:** Contains **meta information** about the page
- **<title>:** Sets the **page title** shown in browser tab or title bar
- **<body>:** Contains all **visible content** (headings, paragraphs, images, etc.)
- **<h1>:** Defines a **large heading**
- **<p>:** Defines a **paragraph**

What is an HTML Element?

- ⇒ An HTML element is a building block of an HTML page, consisting of a **start tag**, **content**, and an **end tag**. It defines the structure and content of parts of a webpage, such as headings, paragraphs, links, images, and more.

Example:

`<h1>My First Heading</h1>`

`<p>My first paragraph.</p>`

Start Tag	Element Content	End Tag
<code><h1></code>	My First Heading	<code></h1></code>
<code><p></code>	My first paragraph.	<code></p></code>

HTML Page Structure

```
<html>

  <head>

    <title>Page title</title>

  </head>

  <body>

    <h1>This is a heading</h1>

    <p>This is a paragraph.</p>

    <p>This is another paragraph.</p>

  </body>

</html>
```

The <!DOCTYPE> Declaration

- Declares the **HTML version** (e.g., HTML5)
- Helps browsers to **render** the page correctly
- Must be the **first line** in the HTML file
- Example: `<!DOCTYPE html>`
- **Not an HTML tag and has no closing tag**

HTML Headings

⇒ HTML headings are used to define **titles or subtitles** on a webpage. They help organize content into sections, making it easier for users and search engines to understand the structure of the page.

- There are **6 levels** of headings: from `<h1>` (highest/most important) to `<h6>` (lowest).
- `<h1>` is typically used for the **main title**, while `<h2>` to `<h6>` are used for **subsections**.
- Headings are **bold** by default and decrease in size from `<h1>` to `<h6>`.
- Proper use of headings improves **readability** and **SEO**.
- Headings should follow a **hierarchical order** for better structure.

Example:	Output:
<pre><!DOCTYPE html> <html> <head> </head> <body> <h1>This is heading 1</h1> <h2>This is heading 2</h2> <h3>This is heading 3</h3> <h4>This is heading 4</h4> <h5>This is heading 5</h5> <h6>This is heading 6</h6> </body> </html></pre>	This is heading 1 This is heading 2 This is heading 3 This is heading 4 This is heading 5 This is heading 6

HTML Paragraphs

⇒ In HTML, a **paragraph** is a block of text defined using the `<p>` tag. It is used to group related sentences and improve the structure and readability of content on a webpage.

- Written using the `<p>...</p>` tag.
- Automatically starts on a **new line**.
- Browsers add **space before and after** each paragraph by default.

Example:	Output:
<pre><p>This is a paragraph.</p> <p>This is another paragraph.</p></pre>	This is a paragraph. This is another paragraph.

HTML Text Formatting Tag

⇒ **Text formatting tags** in HTML are used to style or emphasize text in various ways, such as making it **bold**, **italic**, **underlined**, or **marked**. They help enhance readability and meaning.

Common Formatting Tags & Uses:

Tag	Purpose	Example
<code></code>	Bold (non-semantic)	<code>Bold Text</code>
<code></code>	Strong importance (semantic)	<code>Important</code>
<code><i></code>	Italic (non-semantic)	<code><i>Italic Text</i></code>
<code></code>	Emphasis (semantic italic)	<code>Emphasized</code>
<code><u></code>	Underlined text	<code><u>Underlined</u></code>
<code><mark></code>	Highlighted/marked text	<code><mark>Marked</mark></code>
<code><small></code>	Smaller-sized text	<code><small>Small Text</small></code>
<code></code>	Deleted/struck-through text	<code>Old Price</code>
<code><ins></code>	Inserted/underlined text	<code><ins>New Text</ins></code>
<code><sub></code>	Subscript (e.g., H ₂ O)	H <code><sub>2</sub></code> O
<code><sup></code>	Superscript (e.g., x ²)	x <code><sup>2</sup></code>

Example:	Output:
<pre> <p>This isboldandimportanttext.</p> <p>This is <i>italic</i> and emphasized text.</p> <p>This is <u>underlined</u> and <mark>highlighted</mark>.</p> <p>Water formula: H<sub>2</sub>O</p> <p>Math: x<sup>2</sup> + y<sup>2</sup></p> </pre>	This is bold and important text. This is <i>italic</i> and <i>emphasized</i> text. This is <u>underlined</u> and highlighted. Water formula: H₂O Math: x² + y²

HTML Links

⇒ **HTML links** allow users to **navigate** between different web pages or sections of a page. They are created using the **<a>** (anchor) tag and are one of the most essential elements of the web.

Basic Syntax:

```
<a href="URL">Link Text</a>
```

- **href attribute** specifies the destination URL.
- **Clickable text** is placed between the opening and closing **<a>** tags.

Types of Links:

Type	Example
External link	<code>Google</code>
Internal link	<code>About Page</code>
Email link	<code>Send Email</code>
Phone link	<code>Call Now</code>
Anchor link	<code>Go to Section 1</code>

Open link in new tab:

Use `target="_blank"`

```
<a href="https://example.com" target="_blank">Visit Example</a>
```

HTML Images

⇒ HTML images are used to embed pictures or graphics into a web page using the `<img>` tag. Images help make web content more engaging and informative.

- The `<img>` tag is **self-closing** (void element); it does **not** have an end tag.
- **Essential attributes:**
 - **src** — specifies the **path or URL** of the image.
 - **alt** — provides **alternative text** shown if the image can't be displayed; important for accessibility.
- Optional attributes:
 - **width** and **height** — control the image size (in pixels or percentage).
- Images can be stored locally (relative path) or hosted online (absolute URL).

Basic Syntax:

```

```

Local Image Syntax:

```

```

Online Image Syntax:

```

```

HTML Lists

⇒ HTML lists **organize content** into **ordered or unordered groups**, making information easier to read and understand.

Types of Lists:

Type	Tag	Description
Ordered List	<code></code>	Displays items in a numbered sequence
Unordered List	<code></code>	Displays items with bullet points
List Item	<code></code>	Defines each item inside <code></code> or <code></code>

Basic Syntax:

Ordered List:	Unordered List:
<pre> First item Second item Third item </pre>	<pre> Apple Banana Cherry </pre>

Example:	Output:
<pre><h3>Shopping List</h3> Milk Bread Eggs <h3>Steps to Make Tea</h3> Boil water Add tea leaves Pour into cup </pre>	Shopping List • Milk • Bread • Eggs Steps to Make Tea 1. Boil water 2. Add tea leaves 3. Pour into cup

HTML Tables

⇒ HTML tables organize data into rows and columns, providing a structured way to display tabular information on a webpage.

Key Tags:

Tag	Description
<code><table></code>	Defines the table container
<code><tr></code>	Table row
<code><th></code>	Table header cell (bold and centered by default)
<code><td></code>	Table data cell (regular cell)

Example:	Output:									
<pre><table border="1"> <tr> <th>Name</th> <th>Age</th> <th>City</th> </tr> <tr> <td>John</td> <td>25</td> <td>New York</td> </tr> <tr> <td>Mary</td> <td>30</td> <td>London</td> </tr> </table></pre>	<table><tr><th>Name</th><th>Age</th><th>City</th></tr><tr><td>John</td><td>25</td><td>New York</td></tr><tr><td>Mary</td><td>30</td><td>London</td></tr></table>	Name	Age	City	John	25	New York	Mary	30	London
Name	Age	City								
John	25	New York								
Mary	30	London								
Notes: ➤ The border attribute adds a border around the ➤ Tables can include captions using <caption> tag (optional). ➤ You can merge cells using colspan and rowspan attributes.										

HTML Forms

⇒ HTML forms collect user input and send it to a server for processing. Forms are essential for interaction, such as logging in, signing up, or submitting data.

Key Tags and Attributes:

Tag	Purpose
<code><form></code>	Container for form elements
<code><input></code>	Various input fields (text, password, etc.)
<code><textarea></code>	Multi-line text input
<code><label></code>	Defines labels for input elements
<code><button></code>	Clickable button
<code><select></code>	Drop-down list
<code><option></code>	Options inside <code><select></code>

Important `<form>` Attributes:

Attribute	Description
<code>action</code>	URL where form data is sent
<code>method</code>	HTTP method for submission (GET or POST)

Example:	Output:
<pre><form action="/submit-form" method="post"> <label for="email">Email:</label> <input type="email" id="email" name="email">

 <label for="pwd">Password:</label> <input type="password" id="pwd" name="password">

 <button type="submit">Login</button> </form></pre>	Email: mihmahmud@gmail.com Password: Login

HTML <input> Tag Types

⇒ The **<input>** element is used to create **interactive controls** in a web form. It has **many types** depending on the kind of data you want to collect.

Common <input> Types and Their Uses:

Type	Purpose	Example
text	Single-line text input	<code><input type="text" name="username"></code>
password	Password input (masked)	<code><input type="password" name="pwd"></code>
email	Email address input with validation	<code><input type="email" name="email"></code>
number	Numeric input	<code><input type="number" name="age"></code>
tel	Telephone number input	<code><input type="tel" name="phone"></code>
url	URL input	<code><input type="url" name="website"></code>
checkbox	Checkbox for multiple selections	<code><input type="checkbox" name="subscribe"></code>
radio	Radio buttons for single choice selection	<code><input type="radio" name="gender" value="male"></code>
submit	Submit button to send form data	<code><input type="submit" value="Send"></code>
reset	Reset button to clear form inputs	<code><input type="reset" value="Clear"></code>
button	General clickable button	<code><input type="button" value="Click Me"></code>
file	File selector for uploading files	<code><input type="file" name="upload"></code>
hidden	Hidden input not visible to user	<code><input type="hidden" name="userid" value="123"></code>
date	Date selector	<code><input type="date" name="birthdate"></code>
color	Color picker	<code><input type="color" name="favcolor"></code>
range	Slider control for numeric range	<code><input type="range" name="volume" min="0" max="10"></code>
datetime-local	Date and time selector (local time)	<code><input type="datetime-local" name="appt"></code>
month	Month and year selector	<code><input type="month" name="monthyear"></code>
time	Time selector	<code><input type="time" name="alarm"></code>
week	Week selector	<code><input type="week" name="weekyear"></code>

Example:	Output:
<pre> <form> <input type="text" name="username" placeholder="Username">
 <input type="password" name="pwd" placeholder="Password">
 <input type="email" name="email" placeholder="Email">
 <input type="checkbox" name="subscribe" value="newsletter"> Subscribe to newsletter
 <input type="radio" name="gender" value="male"> Male
 <input type="radio" name="gender" value="female"> Female
 <input type="submit" value="Submit"> </form> </pre>	

HTML Non-Semantic Elements

➔ **Non-semantic HTML elements** do **not** describe their content's meaning or role in the webpage. They are used mainly for **layout and styling purposes** but offer **no meaning to browsers or search engines**. Non-semantic elements are **generic containers**.

Common Non-Semantic Elements:

Tag	Purpose
<code><div></code>	Generic block-level container
<code></code>	Generic inline container

Usage:

- `<div>` is commonly used to group content into sections (e.g., header, content, sidebar).
- `<span>` is used to apply styles or manipulate **inline content**.

Example:
<pre> <div> <h2>This is a div container</h2> <p>This is some content inside a div.</p> </div> <p>This is a highlighted word using span.</p> </pre>

HTML Semantic Elements

⇒ **Semantic elements** in HTML clearly describe their **meaning** in both the **browser** and the **developer's perspective**. Unlike non-semantic elements (like `<div>` or `<span>`), semantic elements define the **structure** and **purpose** of the content inside them.

- Improves **code readability**
- Enhances **SEO (Search Engine Optimization)**
- Boosts **accessibility** for screen readers
- Makes layout and content **easier to maintain**.

Common Semantic Elements:

Tag	Purpose
<code><header></code>	Defines the header of a section or page
<code><nav></code>	Defines a navigation bar or links
<code><main></code>	Represents the main content of the document
<code><section></code>	Defines a generic section of content
<code><article></code>	Self-contained, independent content
<code><aside></code>	Sidebar content, related but separate
<code><footer></code>	Defines the footer of a section or page
<code><figure></code>	Used to wrap media (image, chart, etc.)
<code><figcaption></code>	Caption for the <code><figure></code> content
<code><details></code>	Used for collapsible content
<code><summary></code>	Visible heading for <code><details></code> section
<code><mark></code>	Highlights text
<code><time></code>	Represents a time or date

Example:

```
<main>
  <header>
    <h1>Welcome to My Website</h1>
  </header>

  <nav>
    <a href="#home">Home</a> |
    <a href="#about">About</a>
  </nav>

  <section>
    <article>
      <h2>Blog Post</h2>
      <p>This is a sample blog post using semantic tags.</p>
    </article>
  </section>

  <aside>
    <p>Related Links or Ads</p>
  </aside>

  <footer>
    <p>Copyright © 2025</p>
  </footer>
</main>
```

Output:

Welcome to My Website

[Home](#) | [About](#)

Blog Post

This is a sample blog post using semantic tags.

Related Links or Ads

Copyright © 2025

HTML Comments

⇒ **HTML comments** are notes written in the code that are **ignored by the browser**. They are useful for explaining code, reminding yourself or others about functionality, or temporarily disabling parts of the code

Basic Syntax:

```
<!-- This is a comment -->
```

Note:

- Anything between `<!--` and `-->` is treated as a comment.
- Comments can span single or multiple lines.

Example:

```
<!-- This is the main header section -->
<h1>Welcome to My Website</h1>

<!-- Temporarily hiding this paragraph
<p>This paragraph is under construction.</p>
-->

<!-- Remember to update the link below -->
<a href="about.html">About Us</a>
```

Key Uses:

- Add **notes or reminders** for developers.
- Explain sections of code for better **readability**.
- Temporarily **disable code** without deleting it.
- Helps in **team collaboration**.

HTML Attributes

⇒ **HTML attributes** provide **additional information** about HTML elements. They are always specified in the **start tag** and usually come in **name-value pairs**.

Common HTML Attributes:

Attribute	Description	Used With
href	Specifies the URL of a link	<code><a></code>
src	Specifies the path of an image	<code></code>
alt	Alternative text for an image	<code></code>
title	Tooltip text shown on hover	All elements
style	Inline CSS to style an element	All elements
id	Unique identifier for an element	All elements
class	Class name for grouping elements (used with CSS/JS)	All elements
width/height	Specifies size of image, table, etc.	<code></code> , <code><table></code> , etc.
type	Specifies the type of input or button	<code><input></code> , <code><button></code>
value	Specifies the default value	<code><input></code> , <code><button></code>
name	Name of the input element (important in forms)	<code><input></code> , <code><form></code>

Example:

```

<a href="https://google.com" title="Go to Google" target="_blank">Google</a>
<p id="intro" class="highlight" style="color:blue;">Welcome!</p>
```

Notes:

- Attributes modify the **behavior**, **appearance**, or **function** of elements.
- Some elements require specific attributes to work correctly (e.g., `src` in `<img>`, `href` in `<a>`).
- **Multiple attributes** can be used in one tag.

HTML Multimedia

- ⇒ **HTML Multimedia** refers to the use of **audio** and **video** content on web pages. HTML5 introduced built-in support for multimedia using `<audio>` and `<video>` tags without needing external plugins.

Key Tags:

Tag	Purpose
<code><audio></code>	Embeds sound/audio content
<code><video></code>	Embeds video content
<code><source></code>	Specifies multiple media file sources
<code>controls</code>	Adds play, pause, and volume controls
<code>autoplay</code>	Automatically starts playing
<code>loop</code>	Repeats media after it ends
<code>muted</code>	Starts media in a muted state

Audio Syntax:

```
<audio controls>
  <source src="audiofile.mp3" type="audio/mpeg">
  Your browser does not support the audio tag.
</audio>
```

Video Syntax:

```
<video width="320" height="240" controls>
  <source src="video.mp4" type="video/mp4">
  Your browser does not support the video tag.
</video>
```

What is a Favicon?

⇒ A **favicon** is the small icon shown in the **browser tab**, **bookmark bar**, and **history** next to the page title. It helps users recognize your website.

Syntax to Add a Favicon:

```
<link rel="icon" href="favicon.ico" type="image/x-icon">
```

Notes:

- **href**: Path to your icon file (usually **.ico**, **.png**, or **.svg**)
- The icon file (**favicon.ico**) should be in the same folder or correctly linked
- **type** attribute is optional but recommended

Example:

```
<head>
  <title>My Website</title>
  <link rel="icon" href="images/favicon.png" type="image/png">
</head>
```

Recommended Sizes:

- **.ico** file: 16x16, 32x32, or 48x48
- **.png** file: 32x32 or 64x64

 Tag in HTML

⇒ The **
** tag in HTML stands for "**line break**". It is used to insert a **line break** in text — just like pressing **Enter** on your keyboard.

Example:

```
<p>
  Hello there!<br>
  This is line 2.<br>
  And this is line 3.
</p>
```

Output:

Hello there!
This is line 2.
And this is line 3.



What is CSS?

⇒ CSS stands for **Cascading Style Sheets**. It is a **stylesheet language** used to **describe the presentation** (layout and appearance) of HTML documents.

Purpose of CSS:

- Separates **content (HTML)** from **design (CSS)**
- Controls how HTML elements are displayed
- Helps create **consistent, responsive, and visually appealing** websites

Why Use CSS?

Without CSS	With CSS
Plain, unstyled content	Visually styled, structured, attractive
Hard to maintain styles	Styles are reusable and easier to manage
Style mixed with content	Style kept separate for cleaner code
	

CSS is Used For:

- Setting colors, fonts, margins, padding, borders
- Positioning elements
- Animations and transitions
- Making layouts responsive

Example:

HTML:	CSS:
<pre><p>This is a paragraph.</p></pre>	<pre><style> p { color: blue; font-size: 18px; } </style></pre>

Ways to Apply CSS

⇒ CSS can be applied in **three different ways** based on how and where you define the styles.

- **Inline CSS** (highest priority)
- **Internal CSS** (medium priority)
- **External CSS** (lowest priority)

1. Inline CSS:

- Applied **directly inside** an HTML element using the **style** attribute.
- Affects **only that specific element**.

Example:	Output:
<pre><p style="color: red; font-size: 18px;">This is inline CSS</p></pre>	This is inline CSS

2. Internal CSS:

- Written **inside the <style> tag** within the <head> section of an HTML document.
- Affects **only the current HTML page**.

Example:	Output:
<pre><head> <style> h1 { color: green; text-align: center; } p { font-size: 16px; color: blue; } </style> </head> <body> <h1>Welcome to My Web Page</h1> <p>This is a simple HTML.</p> </body></pre>	Welcome to My Web Page This is a simple HTML.

3. External CSS:

- CSS is written in a **separate file** with **.css** extension (e.g., **style.css**).
- Linked using **<link>** tag inside the **<head>** of the HTML file.
- Used for **styling multiple web pages consistently**.

Example:	
HTML:	CSS:
<pre><head> <link rel="stylesheet" href="style.css"> </head> <body> <h1>Welcome to My Web Page</h1> <p>This is a simple HTML.</p> </body></pre>	<pre>h1 { color: blueviolet; text-align: center; } p { font-size: 16px; color: blue; }</pre>
Output:	
Welcome to My Web Page This is a simple HTML.	

Notes:

- Best practice for all modern web development
- Cleaner and more maintainable

CSS Syntax

⇒ CSS syntax is the way CSS rules are written to define how HTML elements should be styled. Each rule consists of a **selector**, a **property**, and a **value**.

Basic Structure:

```
selector {
  property: value;
}
```

- **Selector:** Identifies the HTML element(s) you want to style
- **Property:** The style attribute you want to change (e.g., color, font-size)
- **Value:** The setting you want to apply to the property (e.g., red, 16px)

Example:

```
h1 {  
  color: blue;  
  font-size: 24px;  
}
```

- **h1** → selector (targets all <h1> elements)
- **color** → property
- **blue** → value
- **font-size: 24px;** → another property-value pair

Multiple Properties:

⇒ You can define multiple properties inside one selector block by separating them with semicolons.

Example:

```
p {  
  color: black;  
  line-height: 1.6;  
  text-align: justify;  
}
```

Notes:

- Always end each property with a ; (semicolon)
- Enclose the property block in {} curly braces
- Use consistent spacing and indentation for readability

CSS Selectors

⇒ CSS **selectors** are patterns used to **select** and **style HTML elements**. They tell the browser **which element(s)** to apply the styles to.

Basic Types of CSS Selectors:

Selector	Description	Example
* (Universal)	Selects all elements	<code>* {margin: 0;}</code>
element	Selects all elements of a specific type	<code>p {color: blue;}</code>
.class	Selects all elements with a specific class	<code>.intro {font-size: 20px;}</code>
#id	Selects a specific element with an ID	<code>#header {background: gray;}</code>
element1, element2	Selects multiple elements	<code>h1, h2 {color: navy;}</code>

Combinator Selectors:

Selector	Meaning	Example
A B	Selects all B inside A (descendant)	<code>div p {color: red;}</code>
A > B	Selects all B that are direct children of A	<code>ul > li {list-style: none;}</code>
A + B	Selects B immediately after A	<code>h1 + p {margin-top: 0;}</code>
A ~ B	Selects all B after A (siblings)	<code>h1 ~ p {color: green;}</code>

Pseudo-element Selectors:

Selector	Purpose	Example
::before	Inserts content before the element content	<code>p::before {content: "Note: ";</code>
::after	Inserts content after the element content	<code>p::after {content: "√";}</code>
::first-line	Styles only the first line of text in an element	<code>p::first-line {font-weight: bold;}</code>

CSS Colors and Units

⇒ CSS supports different ways to define colors. You can use:

1. Color Names: Predefined color names	2. HEX Values: Begins with #, followed by 3 or 6 hexadecimal digits
<pre>h1 { color: red; }</pre>	<pre>p { color: #00ff00; /* Bright green */ }</pre>
3. RGB Values: Defines colors using Red, Green, Blue. Range: 0 to 255	4. RGBA Values: Same as RGB but includes Alpha (opacity). Range of Alpha: 0 to 1
<pre>div { color: rgb(255, 0, 0); /* Red */ }</pre>	<pre>span { color: rgba(0, 0, 255, 0.5); }</pre>

CSS Units

Absolute Units		Relative Units	
Unit	Description	Unit	Description
px	Pixels (most common)	%	Relative to parent element
cm	Centimeters	em	Relative to the font-size of the element
mm	Millimeters	rem	Relative to the font-size of the root element (<html>)
in	Inches	vw	Relative to 1% of the viewport width
pt	Points (1 pt = 1/72 inch)	vh	Relative to 1% of the viewport height
pc	Picas (1 pc = 12 pt)	vmin	Smaller of vw or vh
		vmax	Larger of vw or vh

Example:

```
body {  
  background-color: #f0f0f0;  
  font-size: 1.2rem;  
}  
  
.container {  
  width: 80%;  
  height: 50vh;  
  padding: 20px;  
}
```

CSS Text Styling Properties

⇒ These properties control the **appearance** and **alignment** of text in HTML elements.

1. color:

- Sets the text color.

Example:	
HTML:	CSS:
<pre><head> <link rel="stylesheet" href="style.css"> </head> <body> <p>This is a simple HTML.</p> </body></pre>	<pre>p { color: blue; }</pre>
Output:	
This is a simple HTML.	

2. text-align:

- Aligns the text inside an element.

Value	Meaning
left	Aligns text to the left
right	Aligns text to the right
center	Centers the text
justify	Aligns text evenly

Example:	Output:
<pre>h1 { text-align: center; }</pre>	This is heading 1

3. text-decoration:

- Adds decoration like underline, overline, or removes it.

Value	Meaning
<code>none</code>	No decoration
<code>underline</code>	Underlines text
<code>line-through</code>	Strikes text
<code>overline</code>	Adds line above

Example:	Output:
<pre>a { text-decoration: none; }</pre>	Google.com

4. text-transform:

- Controls letter casing.

Value	Meaning
<code>uppercase</code>	Converts to UPPERCASE
<code>lowercase</code>	Converts to lowercase
<code>capitalize</code>	Capitalizes each word

Example:	Output:
<pre>h2 { text-transform: uppercase; }</pre>	THIS IS HEADING 2

5. letter-spacing & word-spacing:

- Adds space between letters or words.

Example:	Output:
<pre>p { letter-spacing: 2px; word-spacing: 5px; }</pre>	This is a simple HTML.

6. line-height:

- Controls the space between lines of text.

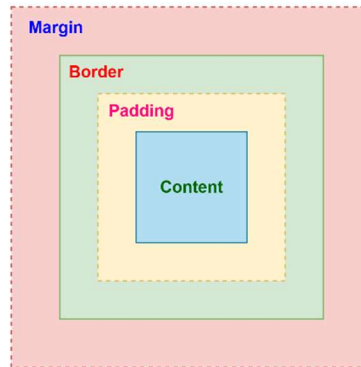
Example:	Output:
<pre>p { line-height: 1.6; }</pre>	This is a simple HTML.

CSS Font Styling Properties

1. font-family: Sets the font of the text. Always provide fallback fonts .	2. font-size: Defines the size of the text.																
<pre>body { font-family: 'Segoe UI', Arial, sans-serif; }</pre>	<pre>h1 { font-size: 32px; }</pre>																
3. font-style: Sets the style of the font.	4. font-weight: Defines the thickness of the text.																
<pre>p { font-style: italic; }</pre>	<pre>span { color: rgba(0, 0, 255, 0.5); }</pre>																
<table><tr><th>Value</th><th>Description</th></tr><tr><td>normal</td><td>Default</td></tr><tr><td>italic</td><td>Italicized text</td></tr><tr><td>oblique</td><td>Slanted text</td></tr></table>	Value	Description	normal	Default	italic	Italicized text	oblique	Slanted text	<table><tr><th>Value</th><th>Description</th></tr><tr><td>normal</td><td>Regular</td></tr><tr><td>bold</td><td>Bold</td></tr><tr><td>100–900</td><td>Numeric weight scale</td></tr></table>	Value	Description	normal	Regular	bold	Bold	100–900	Numeric weight scale
Value	Description																
normal	Default																
italic	Italicized text																
oblique	Slanted text																
Value	Description																
normal	Regular																
bold	Bold																
100–900	Numeric weight scale																

CSS Box Model

- ⇒ The **CSS Box Model** is a fundamental concept that describes how elements are **displayed and spaced** on a web page.
- Every HTML element is considered a **box** composed of:



Parts of the Box Model:

Part	Description
Content	Actual text/image inside the box
Padding	Space between content and border
Border	Wraps the padding and content; thickness/color customizable
Margin	Space outside the border (separates element from others)

Example:	Output:
<pre>div { width: 200px; padding: 20px; border: 5px solid blue; margin: 30px; }</pre>	The output shows a rectangular box with a thick blue border. Inside the box, the text 'Box Model' is centered. The box is surrounded by a white space, representing the margin.
Total width = width + left/right padding + left/right border + left/right margin = 200 + 40 + 10 + 60 = 310px	

Box-sizing Property:

⇒ Controls how the total size of the box is calculated.

Value	Description
<code>content-box</code>	Default. Width/height apply only to content; padding and border added .
<code>border-box</code>	Width/height include content + padding + border (recommended for layout).

Example:	Output:
<pre>.card { width: 300px; padding: 10px; border: 1px solid #ccc; margin: 20px; box-sizing: border-box; }</pre>	 Sample Image Card Title This is a sample card component with an image, title, and description.

CSS Positioning and Layout

⇒ CSS **positioning** is used to place HTML elements **precisely** on the webpage. It defines **how elements are laid out**, stacked, and behave in the document flow.

position Property Values:

Value	Description
<code>static</code>	Default. Elements follow the normal document flow.
<code>relative</code>	Moves element relative to its normal position .
<code>absolute</code>	Positions element relative to the nearest positioned ancestor (not static).
<code>fixed</code>	Positions element relative to the browser window (doesn't scroll).
<code>sticky</code>	Switches between <code>relative</code> and <code>fixed</code> based on scroll position.

Visual Representation of CSS **position** Property Values

Default Position: static

The default position for an element is **position: static**, which doesn't have any effect on the element.

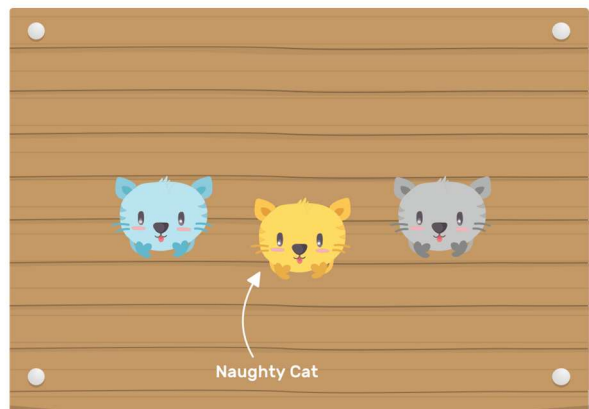
Consider the figure where all the cats are sitting together:



A Relatively Positioned Element:

An element with **position: relative** stays in its place but can be moved using **top**, **bottom**, **left**, or **right**.

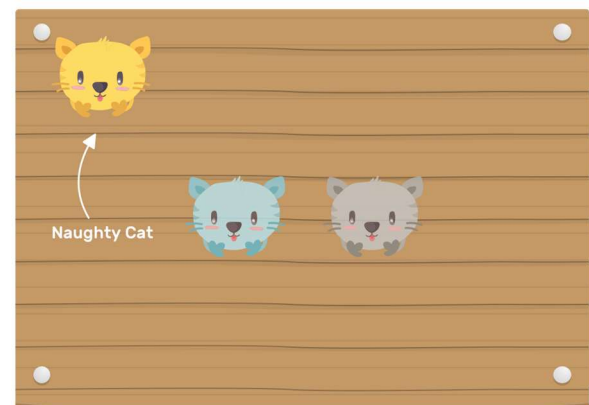
In the picture, a naughty cat has moved down without permission.



An Absolutely Positioned Element:

An element with **position: absolute** moves based on its nearest positioned parent, not its original place. You can use **top**, **bottom**, **left**, or **right** to move it.

In the picture, a naughty cat ran away to the top-left corner without permission.



A Fixed Positioned Element:

An element with `position: fixed` stays in the same place on the screen, even when you scroll. It moves using `top`, `bottom`, `left`, or `right`, based on the viewport (the screen). But if its parent has `transform`, `filter`, or `perspective`, that parent becomes the reference.

In the picture, the cat has escaped to the top-left corner of the viewport. This is where `position: fixed` become handy.

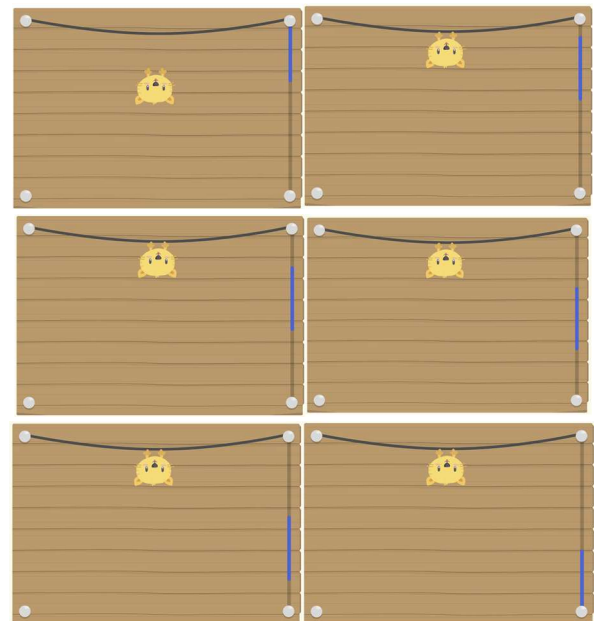


Sticky Positioning

The food is in the center of the wooden box, and the cat wants to stick to the rope at the top. This is where `position: sticky` helps.

With `position: sticky`, the cat stays in place at the top center while you scroll inside the wooden box. Try scrolling to see how it sticks!

```
.wood-wrapper {  
  position: relative;  
}  
  
.the-cat {  
  position: sticky;  
  top: 0;  
}
```



Top, Right, Bottom, Left:

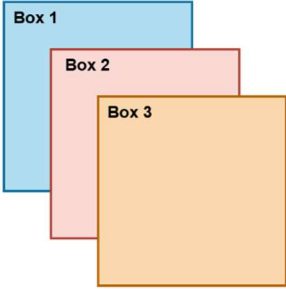
⇒ These properties adjust the element's position **when** `position` is not `static`.

Example:

```
.box {  
  position: relative;  
  top: 10px;  
  left: 20px;  
}
```


z-index:

- Controls the **stacking order** of elements (which appears in front).
- Higher value = more front.

Example:	Output:
<pre>.box1 { z-index: 1; } .box2 { z-index: 2; } .box3 { z-index: 3; }</pre>	

CSS Layout Techniques

1. display Property:

Value	Effect
block	Element takes full width
inline	Element takes only the space it needs
inline-block	Like inline but allows box styling
none	Hides the element
flex	Enables Flexbox layout
grid	Enables CSS Grid layout

2. float and clear:

→ Used to wrap content around elements (e.g., images).

Example:	Output:
<pre> box { float: right; clear: right; } p { clear: left; } </pre>	

3. Flexbox:

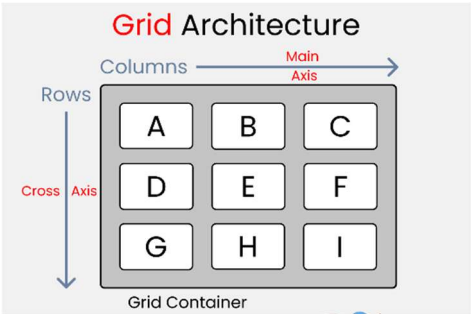
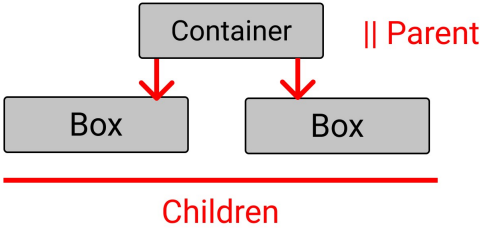
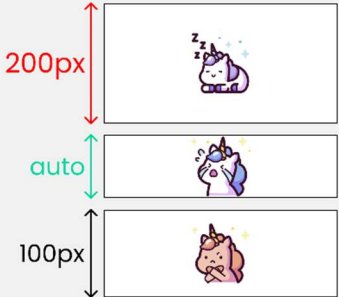
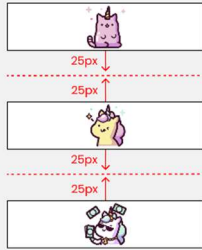
- Powerful 1D layout system.
- Key properties:
 - **display: flex**
 - **justify-content**
 - **align-items**
 - **flex-direction**

Example:	Output:
<pre> .container { display: flex; justify-content: space-between; align-items: center; } </pre>	

justify-content	align-items
flex-direction	flex-wrap

4. CSS Grid:

→ 2D layout system (rows + columns)

CSS Grid Architecture:	Relationship between parent and child classes:
Grid Architecture  Grid Container	Flow 
The grid-template-columns property:	
grid-template-columns : 200px auto 100px; 	grid-template-columns : repeat(3, 1fr); 
The grid-template-row property:	
grid-template-rows : 200px auto 100px 	grid-template-rows : repeat(3, 1fr); 
The row-gap property:	The column-gap property:
row-gap: 50px  Red Dotted lines are called -> grid lines	column-gap: 50px  Red Dotted lines are called -> grid lines

The justify-items property:

	justify-items -> X Axis	justify-items -> X Axis
Start		
end		

The align-items property:

align-items -> Y Axis	
Start	
end	
center	
stretch	

The justify-content property:

justify-content -> X Axis	justify-content -> X Axis

The align-content property:

align-content -> Y Axis	align-content -> Y Axis

The justify-self property:	The align-self property:
justify-self -> X Axis	align-self -> Y Axis
Shorthand for CSS Grid Properties:	
place-content:	place-items:
place-content : align-content / justify-content ↑ ↑ Y-axis X-axis	place-items : align-items / justify-items ↑ ↑ Y-axis X-axis
place-self:	grid-template:
place-self : align-self / justify-self ↑ ↑ Y-axis X-axis	grid-template : grid-template-rows / grid-template-columns
gap/grid-gap:	
gap : row-gap column-gap;	

Example:	Output:
<pre><code>.container { display: grid; grid-template-columns: 1fr 2fr; grid-template-rows: 2fr 1fr; gap: 10px; }</code></pre>	

CSS Layout Techniques

→ CSS Background Properties:

⇒ Used to style the **background** of an element (color, image, position, size, etc.).

1. background-color:

⇒ Sets the background color.

Example:	Output:
<pre>div { background-color: lightblue; }</pre>	Box 1 Box 2 Box 3

2. background-image:

⇒ Sets an image as the background.

Example:	Output:
<pre>div { background-image: url("background.jpg"); }</pre>	Box 1 Box 2 Box 3

3. background-repeat:

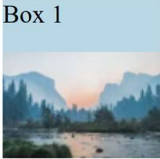
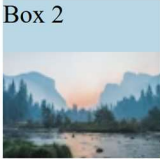
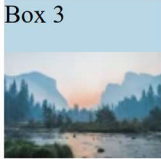
⇒ Controls how the background image repeats.

Value	Description
repeat	Repeats both horizontally and vertically (default)
no-repeat	Image appears once only
repeat-x	Repeats horizontally only
repeat-y	Repeats vertically only

Example:	Output:
<pre>div { background-repeat: no-repeat; }</pre>	Box 1 Box 2 Box 3

4. background-position:

⇒ Defines the starting position of the background image.

Example:	Output:
<pre>div { background-position: center bottom; }</pre>	Box 1  Box 2  Box 3 

5. background-size:

⇒ Specifies the size of the background image.

Value	Description
auto	Default size
cover	Scales image to cover element
contain	Scales image to fit element

Example:	Output:
<pre>div { background-size: cover; }</pre>	Box 1  Box 2  Box 3 

background (shorthand):

Example:	Output:
<pre>div { background: url("bg.jpg") no-repeat center center / cover; }</pre>	Box 1  Box 2  Box 3 

CSS Borders

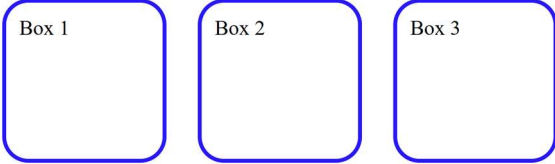
⇒ Used to add borders around elements.

1. border-width, border-style, border-color:

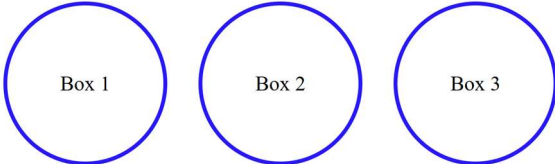
Example:	Output:
<pre>div { border-width: 3px; border-style: dashed; border-color: blue; }</pre>	

2. border-radius:

⇒ Rounds the corners of the element.

Example:	Output:
<pre>div { border-radius: 20px; }</pre>	

⇒ We can create a circle by using **border-radius: 50%**.

Example:	Output:
<pre>div { border-radius: 50%; }</pre>	

2. border- shadow:

⇒ Applies shadow around an element's box.

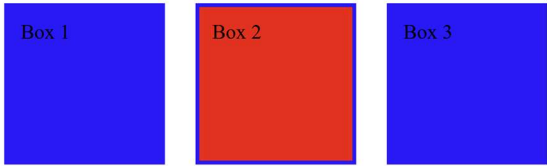
Example:	Output:
<pre>div { box-shadow: 0 4px 8px red; }</pre>	

CSS Transitions and Animations

CSS Transitions

⇒ Transitions make CSS property changes happen **smoothly over time**, instead of instantly.

1. transition (shorthand property):

Example:	Output:
<pre>div { background-color: blue; transition: background-color 0.5s ease; } div:hover { background-color: red; }</pre>	

⇒ Breakdown of Syntax:

transition: property duration timing-function delay;		
Part	Example	Description
property	background-color	CSS property to animate
duration	0.5s	Time it takes (in seconds or milliseconds)
timing-function	ease	Controls the speed curve
delay	0s	Delay before animation starts

⇒ Common timing-function values:

- **linear** – same speed from start to end
- **ease** – slow start, fast, then slow end
- **ease-in** – slow start
- **ease-out** – slow end
- **ease-in-out** – slow start and end

Note: For a better understanding of transitions, **visit:** <https://web.dev/learn/css/transitions>

Want cool transition effects? **Visit this site:** <https://www.transition.style/#in:circle:hesitate>

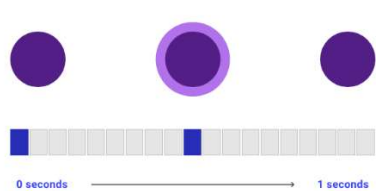
CSS Animations

⇒ Animations define **keyframes** to create more complex movements or effects.

1. @keyframes Rule	2. Applying an Animation
<pre>@keyframes slideRight { from { transform: translateX(0px); } to { transform: translateX(200px); } }</pre>	<pre>div { background: green; animation: slideRight 2s ease-in-out infinite; }</pre>

Animation Properties:

Property	Description
animation-name	Name of the keyframes rule
animation-duration	How long the animation runs
animation-timing-function	Speed curve
animation-delay	Delay before it starts
animation-iteration-count	Number of times it repeats (infinite or number)
animation-direction	normal , reverse , alternate , etc.

Example:	Output:
<pre>@keyframes pulse { 0% { transform: scale(1); } 50% { transform: scale(1.1); } 100% { transform: scale(1); } } div { animation: pulse 1s infinite; }</pre>	
Note: ⇒ For a better understanding of Animations, visit: https://prismic.io/blog/css-animation-examples ⇒ Want cool Animations effects? Visit this site: https://animate.style/	

Responsive CSS

⇒ **Responsive CSS** ensures that a website **adapts** its layout and appearance across different screen sizes and devices—like mobiles, tablets, laptops, and desktops.

Why Use Responsive Design?

- Improves **user experience**
- Ensures **accessibility** on all devices
- Makes the layout **fluid and flexible**
- Reduces the need for a separate mobile site

Key Concepts in Responsive CSS

1. Viewport Meta Tag (HTML)

→ Use this in the **<head>** of your HTML to control layout scaling:

```
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
```

2. Media Queries

→ Allows you to **apply CSS rules** only when **certain conditions** (like screen width) are met.

```
@media (max-width: 768px) {
  body {
    background-color: lightgray;
  }
}
```

Common Breakpoints	Device Type
max-width: 480px	Mobile (portrait)
max-width: 768px	Tablets & small screens
max-width: 1024px	Desktops & laptops

3. Relative Units

→ Use **flexible units** for layout and font sizes:

- **%, em, rem, vw, vh**
- Avoid using fixed **px** for responsive elements

```
.container {  
  width: 80%;  
  padding: 2vw;  
}
```

4. Flexible Layouts with Flexbox/Grid

- **flex-wrap: wrap;** helps in making flex containers responsive
- CSS Grid allows auto-fitting with **minmax()** and **auto-fit**

```
.container {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));  
}
```

5. Responsive Images

- **max-width: 100%** → Prevents the image from overflowing its container.
- **height: auto** → Keeps the image's aspect ratio.

```

```

Best Practices

- Always test on multiple devices/screen sizes
- Start with **mobile-first** design using **min-width** media queries
- Keep layout **fluid** using **%** or **vw/vh** instead of fixed sizes

Common CSS Examples

1. Center Text	2. Center a Div Horizontally
<pre>.center-text { text-align: center; }</pre>	<pre>.center-div { width: 300px; margin: 0 auto; }</pre>
3. Responsive Image	4. Rounded Corners and Shadow
<pre>img { width: 100%; height: auto; }</pre>	<pre>.card { border-radius: 10px; box-shadow: 0 4px 10px rgba(0, 0, 0, 0.2); }</pre>
5. Button Hover Effect	6. Sticky Navbar
<pre>button { background-color: blue; color: white; padding: 10px 20px; border: none; transition: 0.3s; } button:hover { background-color: darkblue; }</pre>	<pre>.navbar { position: sticky; top: 0; background-color: #fff; padding: 10px; z-index: 999; }</pre>
7. Text with Ellipsis (when too long)	8. Responsive Grid Layout
<pre>.text-ellipsis { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }</pre>	<pre>.grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 20px; }</pre>
8. Flexbox Centering	10. Fade In Animation
<pre>.flex-center { display: flex; justify-content: center; align-items: center; height: 100vh; }</pre>	<pre>@keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } } .fade-in { animation: fadeIn 1s ease-in; }</pre>

GIT & GIT HUB

What is Git?

- Git is a **distributed version control system**.
- Helps track **changes in code**, collaborate with others, and revert back if needed.
- Developed by **Linus Torvalds** in 2005.

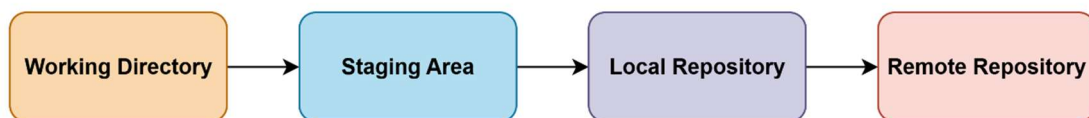
What is GitHub?

- GitHub is a **cloud-based hosting service** for Git repositories.
- Offers GUI tools, issue tracking, pull requests, and team collaboration.

Basic Git Terminology

Term	Description
Repository	A directory where your project and its version history live
Commit	A snapshot of your code at a given time
Branch	A separate line of development
Merge	Combines changes from one branch into another
Clone	Downloads a remote repository to your local machine
Push	Upload local changes to the remote repository
Pull	Download latest changes from the remote repo
Staging Area	Where changes are prepared before committing
HEAD	Refers to the current branch or commit you're working on

Git Workflow



Essential Git Commands

Command	Description
<code>git init</code>	Initialize a new Git repository
<code>git clone <url></code>	Clone a repo from GitHub
<code>git status</code>	Show changes in working directory
<code>git add <file></code>	Stage a file for commit
<code>git commit -m "message"</code>	Commit changes with a message
<code>git log</code>	View commit history
<code>git branch</code>	List branches
<code>git branch <branch-name></code>	Create a new branch
<code>git checkout <branch-name></code>	Switch to another branch
<code>git merge <branch-name></code>	Merge a branch into current branch
<code>git pull</code>	Fetch and merge changes from remote
<code>git push</code>	Upload local changes to GitHub

Useful GitHub Features

Feature	Purpose
Pull Request	Request to merge code into another branch
Fork	Copy someone else's repo to your account
Stars	Like or bookmark a project
Issues	Report bugs or suggest enhancements
Actions	Automate workflows (CI/CD, testing, etc.)

GitHub Workflow

1. Create a **repository** on GitHub
2. **Clone** it to your local system
3. Make changes in code
4. Use `git add`, `git commit`, `git push`
5. Changes appear on GitHub

Git is the free and open source distributed version control system that's responsible for everything GitHub related that happens locally on your computer. This cheat sheet features the most important and commonly used Git commands for easy reference.

INSTALLATION & GUIs

With platform specific installers for Git, GitHub also provides the ease of staying up-to-date with the latest releases of the command line tool while providing a graphical user interface for day-to-day interaction, review, and repository synchronization.

GitHub for Windows

<https://windows.github.com>

GitHub for Mac

<https://mac.github.com>

For Linux and Solaris platforms, the latest release is available on the official Git web site.

Git for All Platforms

<http://git-scm.com>

SETUP

Configuring user information used across all local repositories

```
git config --global user.name "[firstname lastname]"
```

set a name that is identifiable for credit when review version history

```
git config --global user.email "[valid-email]"
```

set an email address that will be associated with each history marker

```
git config --global color.ui auto
```

set automatic command line coloring for Git for easy reviewing

SETUP & INIT

Configuring user information, initializing and cloning repositories

```
git init
```

initialize an existing directory as a Git repository

```
git clone [url]
```

retrieve an entire repository from a hosted location via URL

STAGE & SNAPSHOT

Working with snapshots and the Git staging area

```
git status
```

show modified files in working directory, staged for your next commit

```
git add [file]
```

add a file as it looks now to your next commit (stage)

```
git reset [file]
```

unstage a file while retaining the changes in working directory

```
git diff
```

diff of what is changed but not staged

```
git diff --staged
```

diff of what is staged but not yet committed

```
git commit -m "[descriptive message]"
```

commit your staged content as a new commit snapshot

BRANCH & MERGE

Isolating work in branches, changing context, and integrating changes

```
git branch
```

list your branches. a * will appear next to the currently active branch

```
git branch [branch-name]
```

create a new branch at the current commit

```
git checkout
```

switch to another branch and check it out into your working directory

```
git merge [branch]
```

merge the specified branch's history into the current one

```
git log
```

show all commits in the current branch's history



INSPECT & COMPARE

Examining logs, diffs and object information

git log

show the commit history for the currently active branch

git log branchB..branchA

show the commits on branchA that are not on branchB

git log --follow [file]

show the commits that changed file, even across renames

git diff branchB..branchA

show the diff of what is in branchA that is not in branchB

git show [SHA]

show any object in Git in human-readable format

TRACKING PATH CHANGES

Versioning file removes and path changes

git rm [file]

delete the file from project and stage the removal for commit

git mv [existing-path] [new-path]

change an existing file path and stage the move

git log --stat -M

show all commit logs with indication of any paths that moved

IGNORING PATTERNS

Preventing unintentional staging or committing of files

```
logs/  
*.notes  
pattern*/
```

Save a file with desired patterns as .gitignore with either direct string matches or wildcard globs.

git config --global core.excludesfile [file]

system wide ignore pattern for all local repositories

SHARE & UPDATE

Retrieving updates from another repository and updating local repos

git remote add [alias] [url]

add a git URL as an alias

git fetch [alias]

fetch down all the branches from that Git remote

git merge [alias]/[branch]

merge a remote branch into your current branch to bring it up to date

git push [alias] [branch]

Transmit local branch commits to the remote repository branch

git pull

fetch and merge any commits from the tracking remote branch

REWRITE HISTORY

Rewriting branches, updating commits and clearing history

git rebase [branch]

apply any commits of current branch ahead of specified one

git reset --hard [commit]

clear staging area, rewrite working tree from specified commit

TEMPORARY COMMITS

Temporarily store modified, tracked files in order to change branches

git stash

Save modified and staged changes

git stash list

list stack-order of stashed file changes

git stash pop

write working from top of stash stack

git stash drop

discard the changes from top of stash stack

GitHub Education

Teach and learn better, together. GitHub is free for students and teachers. Discounts available for other educational uses.

✉ education@github.com
🌐 education.github.com

JAVASCRIPT

INTRODUCTION TO JAVASCRIPT

Introduction

➤ What is JavaScript?

- ⇒ **JavaScript (JS)** is a high-level, dynamic, interpreted programming language.
- ⇒ Primarily used to make web pages **interactive** and **dynamic**.
- ⇒ Runs in the browser but can also run on servers using **Node.js**.

➤ Features of JavaScript?

- **Interpreted**: No need for compilation, runs directly in the browser.
- **Dynamic Typing**: Data types are determined at runtime.
- **Event-Driven**: Can respond to user actions (clicks, keystrokes, etc.).
- **Prototype-Based**: Uses prototypes for inheritance instead of classical classes.
- **Cross-Platform**: Works on all major browsers and devices.

Difference Between JavaScript, Java, and ECMAScript

Feature	JavaScript	Java	ECMAScript
Type	Scripting language	Object-oriented programming	Standard specification for JS
Execution	Runs in browsers & Node.js	Runs in JVM	Not a language, but a standard
Syntax	Loosely typed	Strongly typed	Defines syntax & rules of JavaScript
Usage	Web interactivity	Enterprise applications	Acts as blueprint for JavaScript

➤ History

- ⇒ Created in **1995** by **Brendan Eich** at Netscape.
- ⇒ Originally called **Mocha**, then **LiveScript**, and finally **JavaScript**.
- ⇒ Standardized as **ECMAScript** in 1997 by **ECMA International**.

➤ Why Learn JavaScript?

- **Essential for Web Development**: Alongside HTML & CSS, it's one of the three core web technologies.
- **Versatile**: Works for frontend, backend, mobile apps, desktop apps, and even IoT.
- **Huge Community & Libraries**: jQuery, React, Vue, Angular, Node.js, Express.js.

Variables

⇒ **Variable** is a name used to **store data** in memory. It acts as a container for values.

➤ Declaring Variables?

Keyword	Scope	Reassignment	Hoisting	Notes
<code>var</code>	Function	Yes	Yes	Old way, avoid in modern JS
<code>let</code>	Block	Yes	No	Preferred for variables that change
<code>const</code>	Block	No	No	Preferred for constants

➤ How to Declare a Variable?

Syntax:	Example Code:
<code>var x = 10; let y = 20; const z = 30;</code>	<code>var x = 10; // function-scoped let y = 20; // block-scoped const z = 30; // cannot be reassigned</code>
Rules for Naming Variables:	
• Must start with a letter or underscore (<code>_</code>) • Cannot start with a digit • Can contain letters, digits, and underscores • Case-sensitive (<code>age</code> and <code>Age</code> are different)	

Data Types

⇒ JavaScript has **primitive types** and **reference types**.

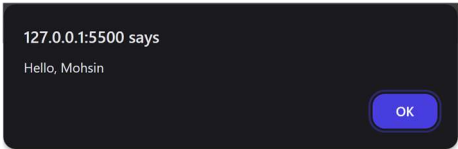
Primitive Types:		
Type	Example	Description
<code>string</code>	<code>"Hello"</code>	Text data
<code>number</code>	<code>42, 3.14</code>	Numbers (integer & float)
<code>boolean</code>	<code>true, false</code>	Logical values
<code>null</code>	<code>null</code>	Intentional empty value
<code>undefined</code>	<code>let x;</code>	Declared but not assigned
<code>symbol</code>	<code>Symbol('id')</code>	Unique identifiers
<code>bigint</code>	<code>123n</code>	Large integers

Example Code:

```
let name = "Alice";    // string
let age = 20;          // number
let isStudent = true;  // boolean
let car = null;        // null
let city;              // undefined
```

Output Methods

Method	Where It Shows	Example
<code>console.log()</code>	Browser console	<code>console.log("Hello")</code>
<code>alert()</code>	Popup box	<code>alert("Warning!")</code>
<code>document.write()</code>	Directly to web page	<code>document.write("Hi")</code>

Example Code: <code>console.log()</code>	Output:
<pre>let name_ = "Mohsin"; console.log("Name:", name_)</pre>	Mohsin
Example Code: <code>alert()</code>	Output:
<pre>let name_ = "Mohsin"; alert("Hello, " + name_);</pre>	
Example Code: <code>document.write()</code>	Output:
<pre>let name_ = "Mohsin"; const country = "Bangladesh"; let age = 21; document.write(`You are from \${country} and \${age} years old.`);</pre>	You are from Bangladesh and 21 years old.

How to Run JavaScript

In HTML:

```
<!-- Inline -->
<script>
  console.log("Hello World");
</script>

<!-- External File -->
<script src="script.js"></script>
```

Tip: Place scripts **before** `</body>` so the page loads faster.

Type Conversion

➔ **Type Conversion** means changing the data type of a value or variable.

Conversion	Example	Output
String → Number	<code>Number("10")</code>	10
Number → String	<code>String(10)</code>	"10"
Boolean → Number	<code>Number(true)</code>	1
Automatic (coercion)	<code>"5" * 2</code>	10

Comments

⇒ **Comments** are notes in the code that are ignored by the browser, used to explain or temporarily disable code.

Type	Syntax	Example
Single-line	<code>//</code>	<code>// This is a comment</code>
Multi-line	<code>/* ... */</code>	<code>/* This is a multi-line comment */</code>

Operators

⇒ Operators are **special symbols** used to perform operations on **values or variables**.

Example Code:	Output:
<pre>let a = 5; let b = 2; let sum = a + b; // + is the operator console.log(sum); // 7</pre>	7

Types of Operators

a) Arithmetic Operators:

⇒ Used for **mathematical calculations**.

Operator	Description	Example	Result
+	Addition	5 + 2	7
-	Subtraction	5 - 2	3
*	Multiplication	5 * 2	10
/	Division	5 / 2	2.5
%	Modulus (remainder)	5 % 2	1
**	Exponentiation	5 ** 2	25
++	Increment (+1)	a++	6
--	Decrement (-1)	b--	1

b) Assignment Operators:

⇒ Used to **assign values** to variables.

Operator	Example	Meaning
=	x = 5	Assign 5 to x
+=	x += 2	x = x + 2
-=	x -= 2	x = x - 2
*=	x *= 2	x = x * 2
/=	x /= 2	x = x / 2
%=	x %= 2	x = x % 2

c) Comparison Operators:

⇒ Used to **compare two values**. Result is **true** or **false**.

Operator	Example	Meaning
<code>==</code>	<code>5 == '5'</code>	Equal (value only)
<code>===</code>	<code>5 === '5'</code>	Equal (value & type)
<code>!=</code>	<code>5 != 2</code>	Not equal (value)
<code>!==</code>	<code>5 !== '5'</code>	Not equal (value & type)
<code>></code>	<code>5 > 2</code>	Greater than
<code><</code>	<code>5 < 2</code>	Less than
<code>>=</code>	<code>5 >= 5</code>	Greater or equal
<code><=</code>	<code>5 <= 2</code>	Less or equal

d) Logical Operators:

⇒ Used for **Boolean logic**.

Operator	Example	Result	Meaning
<code>&&</code>	<code>true && false</code>	<code>false</code>	AND — true only if both are true
<code> </code>	<code>true false</code>	<code>true</code>	OR — true if at least one is true
<code>!</code>	<code>!true</code>	<code>false</code>	NOT — flips the Boolean value

In JavaScript:

- `&&` stops checking if the first value is **false** (short-circuiting).
- `||` stops checking if the first value is **true**.
- `!` works on truth/falsie values, not just `true/false`.

e) String Operators:

⇒ Mainly **concatenation**.

Example Code:	Output:
<pre>let firstName = "Mohsin"; let lastName = "ELI"; let fullName = firstName + " " + lastName; console.log(fullName);</pre>	Mohsin ELI

How to See the Data Type

⇒ `typeof` → returns **data type**.

Example Code:	Output:
<pre>let x = 10; let str = "Hello, World!"; console.log(typeof x); // number console.log(typeof str); // string</pre>	<pre>number string</pre>

Keywords

➔ **Keywords** are reserved words in JavaScript. We cannot use them as variable names or identifiers.

List of Common JavaScript Keywords:

break	case	catch	class	const	continue
debugger	default	delete	do	else	export
extends	finally	for	function	if	import
in	instanceof	new	return	super	switch
this	throw	try	typeof	var	void
while	with	yield	let	enum	await

✂ **Problem1:** Write a JavaScript program that assign values to variables num1, num2 and perform addition operations. Also print the results.

Code:	Output:
<pre>let num1 = 10; let num2 = 5; let sum = num1 + num2; console.log("Sum:", sum);</pre>	Sum: 15

✂ **Problem2:** Write a JavaScript program that assign values to variables num1, num2 and perform subtraction operations. Also print the results.

Code:	Output:
<pre>let num1 = 10; let num2 = 5; let sub = num1 - num2; console.log("Sub:", sub);</pre>	Sub: 5

✂ **Problem3:** Write a JavaScript program that assign values to variables num1, num2 and perform multiplication & division operations. Also print the results.

Code:	Output:
<pre>let num1 = 10; let num2 = 5; let mult = num1 * num2; let div = num1 / num2; console.log("Mult:", mult); console.log("Div:", div);</pre>	Mult: 50 Div: 2.0

Input in JavaScript

➔ The `prompt()` shows a pop-up box that asks the user for input.

Syntax:

```
let variable = prompt("Enter something: ")
```

Example Code:

Example code:	Output:
<pre>let name = prompt("Enter your name:"); console.log("Hello, " + name);</pre>	Enter your name: Mohsin Ibna Hossain Hello, Mohsin Ibna Hossain
Convert input to other types:	Output:
<pre>let input = prompt("Enter a number:"); console.log("Type:", typeof input); let num = Number(input); console.log("Number:", num); console.log("Type:", typeof num);</pre>	Enter a number: 23 Type: string Number: 23 Type: number

➔ **Note:** In JavaScript, when you take user input (usually with `prompt()` in the browser), it comes in as a **string**, so you need to convert it to other types like **number** or **boolean** if needed.

Conversion Type	Method	Example	Result
String → Number (integer)	<code>parseInt()</code>	<code>parseInt("42")</code>	42
String → Number (float)	<code>parseFloat()</code>	<code>parseFloat("3.14")</code>	3.14
String → Number (auto)	<code>Number()</code>	<code>Number("42.5")</code>	42.5
Number → String	<code>.toString()</code> OR <code>String()</code>	<code>(42).toString()</code>	"42"
String → Boolean	<code>Boolean()</code>	<code>Boolean("Hello")</code>	true
Boolean → Number	<code>Number(true)</code>	<code>Number(true)</code>	1

Practice Problem

🔗 **Problem1:** Write a Program to input 2 numbers & print their sum.

Code:	Output:
<pre>let num1 = Number(prompt("Enter number 1:")); let num2 = Number(prompt("Enter number 2:")); let sum = num1 + num2; console.log("Sum:", sum);</pre>	Enter number 1: 10 Enter number 2: 20 Sum: 30

🔗 **Problem2:** Write a JavaScript program to take five float variables as input from the user. Find out the summation and average of five numbers

Code:	Output:
<pre>let num1 = parseFloat(prompt("Enter number 1:")); let num2 = parseFloat(prompt("Enter number 2:")); let num3 = parseFloat(prompt("Enter number 3:")); let num4 = parseFloat(prompt("Enter number 4:")); let num5 = parseFloat(prompt("Enter number 5:")); let summ = num1 + num2 + num3 + num4 + num5; let average = summ / 5; console.log("Summation:", summ); console.log("Average:", average);</pre>	Enter number 1: 10 Enter number 2: 3 Enter number 3: 7 Enter number 4: 6 Enter number 5: 8 Summation: 34 Average: 6.8

CONDITIONAL STATEMENTS

Conditional Statements

➔ **Conditional statements** let you execute different blocks of code based on conditions.

Common String Operations:

if Statement:	
<pre>let age = 20; if (age >= 18) { console.log("You are an adult"); }</pre>	
if...else Statement:	if...else if...else Statement:
<pre>let age = 16; if (age >= 18) { console.log("You are an adult"); } else { console.log("You are a minor"); }</pre>	<pre>let score = 85; if (score >= 90) { console.log("Grade: A"); } else if (score >= 80) { console.log("Grade: B"); } else { console.log("Grade: C or below"); }</pre>
Notes: • Conditions use comparison operators (==, !=, >, <, >=, <=)	

Nested Conditional Statements

➔ A **nested conditional** means using one if or else block **inside another**.

It allows for **more complex decision making**.

Nested if:	Output:
<pre>let age = 25; let hasID = true; if (age >= 18) { if (hasID) { console.log("You can enter"); } else { console.log("ID required"); } } else { console.log("You are too young"); }</pre>	You can enter

Practice Problem

✂ **Problem1:** Write a JavaScript program to check whether a number entered by the user is **odd** or **even**, and print the result.

Code:	Output:
<pre>let num = Number(prompt("Enter a number:")); if (num % 2 === 0) { console.log("Even"); } else { console.log("Odd"); }</pre>	Enter a number: 10 Even
	Enter a number: 5 Odd

✂ **Problem2:** Write a JavaScript program to **grade students based on their marks** using the following criteria:

- If marks are **90 or above**, grade = "A"
 - If marks are **80 to 89**, grade = "B"
 - If marks are **70 to 79**, grade = "C"
 - If marks are **below 70**, grade = "D"
- Then print the grade.

Code:	Output:
<pre>let marks = Number(prompt("Enter student's marks:")); let grade; if (marks >= 90) { grade = "A"; } else if (marks >= 80) { grade = "B"; } else if (marks >= 70) { grade = "C"; } else { grade = "D"; } console.log("Grade:", grade);</pre>	Enter student's marks: 73 Grade: C

✂ **Problem3:** Write a JavaScript program to input three numbers from the user and print the **greatest** among them.

Code:	Output:
<pre>let a = Number(prompt("Enter first number:")); let b = Number(prompt("Enter second number:")); let c = Number(prompt("Enter third number:")); let greatest; if (a >= b && a >= c) { greatest = a; } else if (b >= a && b >= c) { greatest = b; } else { greatest = c; } console.log("The greatest number is:", greatest);</pre>	Enter first number: 10 Enter second number: 20 Enter third number: 30 The greatest number is: 30
	Enter first number: 15 Enter second number: 10 Enter third number: 5 The greatest number is: 15
	Enter first number: 7 Enter second number: 21 Enter third number: 14 The greatest number is: 21

✂ **Problem6:** Write a JavaScript program to check whether a number is divisible by 2, 5, and 11. Also, demonstrate the use of both OR and AND operators in the program.

Code:	Output:
<pre>let num = Number(prompt("Enter a number:")); if (num % 2 === 0 && num % 5 === 0 && num % 11 === 0) { console.log("Number is divisible by 2, 5, and 11."); } else { console.log("Number is NOT divisible by 2, 5, and 11."); } if (num % 2 === 0 num % 5 === 0 num % 11 === 0) { console.log("Number is divisible by at least one of 2, 5, or 11."); } else { console.log("Number is not divisible by 2, 5, or 11."); }</pre>	Enter a number: 110 number is divisible by 2, 5, and 11. number is divisible by at least one of 2, 5, or 11.
	Enter a number: 37 number is NOT divisible by 2, 5, and 11. number is not divisible by 2, 5, or 11.

LOOPS

Loops

➔ **Loops** are used to execute a block of code repeatedly.

Types of Loops:

for Loop: iterate over a sequence	while Loop: runs while a condition is True
<pre>for (let i = 1; i <= 5; i++) { console.log(i); }</pre>	<pre>let i = 1; while (i <= 5) { console.log(i); i++; }</pre>
do...while Loop: Runs the code once before checking the condition.	for...of Loop: Loops through iterable objects like arrays or strings.
<pre>let i = 1; do { console.log(i); i++; } while (i <= 5);</pre>	<pre>let colors = ["red", "green", "blue"]; for (let color of colors) { console.log(color); }</pre>
for...in Loop: Loops through object properties .	
<pre>let person = {name: "John", age: 30}; for (let key in person) { console.log(key + ": " + person[key]); }</pre>	
break Statement: Stops the loop immediately.	continue Statement: Skips the current iteration and moves to the next.
<pre>for (let i = 1; i <= 10; i++) { if (i === 5) { break; } console.log(i); }</pre>	<pre>for (let i = 1; i <= 5; i++) { if (i === 3){ continue; } console.log(i); }</pre>

Practice Problem

🔗 **Problem1:** Write a JavaScript program to **print numbers from 1 to 5** using a **while** loop.

Code:	Output:
<pre>let i = 1; while (i <= 5) { console.log(i); i++; }</pre>	1 2 3 4 5

🔗 **Problem2:** Write a JavaScript program to **print numbers from 5 to 1** using a **while** loop.

Code:	Output:
<pre>let i = 5; while (i >= 1) { console.log(i); i--; }</pre>	5 4 3 2 1

🔗 **Problem3:** Write a JavaScript program to input a number **n** and print its **multiplication table** using a **while** loop.

Code:	Output:
<pre>let n = parseInt(prompt("Enter a number:")); let i = 1; while (i <= 10) { console.log(n + " x " + i + " = " + (n * i)); i++; }</pre>	Enter a number: 11 11 x 1 = 11 11 x 2 = 22 11 x 3 = 33 11 x 4 = 44 11 x 5 = 55 11 x 6 = 66 11 x 7 = 77 11 x 8 = 88 11 x 9 = 99 11 x 10 = 110

🔗 **Problem4:** Write a JavaScript program to **input a number n** and print the **sum of all numbers from 1 to n** using a **for** loop.

Code:	Output:
<pre>let n = parseInt(prompt("Enter a number:")); let sum = 0; for (let i = 1; i <= n; i++) { sum += i; } console.log("The sum of 1 to " + n + " is: " + sum);</pre>	Enter a number: 10 The sum of 1 to 10 is: 55

✂ **Problem4:** Write a JavaScript program that takes numbers as input from the user using a `while` loop. The loop should **stop if the user enters -1**. Print the **sum** of all entered numbers (excluding -1).

Code:	Output:
<pre>let total = 0; while (true) { let num = parseInt(prompt("Enter a number (-1 to stop):")); if (num === -1) { break; } total += num; } console.log("Total sum is: " + total);</pre>	Enter a number (-1 to stop): 10 Enter a number (-1 to stop): 5 Enter a number (-1 to stop): 3 Enter a number (-1 to stop): 2 Enter a number (-1 to stop): 1 Enter a number (-1 to stop): -1 Total sum is: 21

✂ **Problem5:** Write a JavaScript program to print all numbers from **1 to 10 in a line**, except for number **5**, using a `for` loop and the `continue` statement.

Code:	Output:
<pre>for (let i = 1; i <= 10; i++) { if (i === 5) { continue; } process.stdout.write(i + " "); }</pre>	1 2 3 4 6 7 8 9 10

STRING

Strings

➔ A **string** is a sequence of characters inside **single quotes** ('), **double quotes** (""), or **backticks** (`).

Example Code:

```
let str1 = "Hello";
let str2 = 'World';
let str3 = `Hello World`; // template literal
```

String Creation:

```
let single = 'Hello';
let double = "Hello";
let backtick = `Hello`; // allows variables & expressions
```

Template Literals: Allow variables and expressions inside strings using `${}`.

```
let name = "John";
let age = 25;
console.log(`My name is ${name} and I am ${age} years old.`);
```

Common String Methods:

Method	Description	Example
<code>toUpperCase()</code>	Converts to uppercase	<code>"hello".toUpperCase()</code> → <code>"HELLO"</code>
<code>toLowerCase()</code>	Converts to lowercase	<code>"HELLO".toLowerCase()</code> → <code>"hello"</code>
<code>charAt(index)</code>	Returns character at index	<code>"Hello".charAt(1)</code> → <code>"e"</code>
<code>indexOf(value)</code>	Finds position of first match	<code>"Hello".indexOf("l")</code> → 2
<code>lastIndexOf(value)</code>	Finds last match	<code>"Hello".lastIndexOf("l")</code> → 3
<code>includes(value)</code>	Checks if contains substring	<code>"Hello".includes("lo")</code> → <code>true</code>
<code>startsWith(value)</code>	Checks if starts with substring	<code>"Hello".startsWith("He")</code> → <code>true</code>
<code>endsWith(value)</code>	Checks if ends with substring	<code>"Hello".endsWith("lo")</code> → <code>true</code>

<code>slice(start, end)</code>	Extracts part of a string	<code>"Hello".slice(1, 4) → "ell"</code>
<code>substring(start, end)</code>	Similar to slice	<code>"Hello".substring(1, 4) → "ell"</code>
<code>substr(start, length)</code>	Extracts part by length	<code>"Hello".substr(1, 3) → "ell"</code>
<code>replace(old, new)</code>	Replaces text	<code>"Hello".replace("He", "Ye") → "Yello"</code>
<code>replaceAll(old, new)</code>	Replaces all matches	<code>"hi hi".replaceAll("hi", "hey") → "hey hey"</code>
<code>trim()</code>	Removes spaces from both ends	<code>" hello ".trim() → "hello"</code>
<code>repeat(n)</code>	Repeats string n times	<code>"Hi".repeat(3) → "HiHiHi"</code>
<code>split(separator)</code>	Converts to array	<code>"a,b,c".split(",") → ["a", "b", "c"]</code>

Common String Properties:

Property	Description	Example
<code>.length</code>	Returns string length	<code>"Hello".length // 5</code>

Escape Characters

Code	Meaning
<code>\'</code>	Single quote
<code>\"</code>	Double quote
<code>\\</code>	Backslash
<code>\n</code>	New line
<code>\t</code>	Tab

Example code:	Output:
<pre>console.log("Hello\nWorld");</pre>	Hello World
Example code:	Output:
<pre>let a = "Hello"; let b = "World"; console.log(a + " " + b); console.log(`\${a} \${b}`);</pre>	Hello World Hello World

Practice Problem

🔗 **Problem1:** Write a JavaScript to input user's first name & print its length.

Code:	Output:
<pre>let first_name = prompt("Enter first name:"); let length = first_name.length; console.log("Length of first name is: " + length);</pre>	Enter first name: Mohsin Length of first name is: 6

🔗 **Problem2:** Write a JavaScript program to initialize a string and find how many times the character '\$' appears in it. text = "He bought a pen for \$5 and a book for \$10, total \$15."

Code:	Output:
<pre>let text = "He bought a pen for \$5 and a book for \$10, total \$15."; let count = 0; for (let i = 0; i < text.length; i++) { if (text[i] === '\$') { count++; } } console.log("The character '\$' appears " + count + " time(s).");</pre>	The character '\$' appears 3 time(s).

🔗 **Problem3:** Prompt the user to enter their full name. Generate a username for them based on the input. Start username with @, followed by their full name and ending with the fullname length. eg: user name = "mohsineli", username should be "@mohsineli9"

Code:	Output:
<pre>const fullName = prompt("full name:"); const username = `@\${fullName}\${fullName.length}`; console.log(`Your username: \${username}`); alert(`Your username is: \${username}`);</pre>	Your username: @mohsineli9

ARRAYS

Arrays

⇒ An **array** is a special variable that can hold **multiple values** in a single name.

- Array elements can be of **different data types**.
- Indexing starts at **0**.

Syntax:	Example:
<pre>let llistVariableName = ["item1", "item2"]</pre>	<pre>let fruits = ["Apple", "Banana", "Mango"];</pre>
Creating Arrays:	
<pre>let arr1 = ["Apple", "Banana", "Mango"]; // literal let arr2 = new Array("Apple", "Banana"); // constructor let arr3 = []; // empty array</pre>	
Accessing Array Elements:	
<pre>let fruits = ["Apple", "Banana", "Mango"]; console.log(fruits[0]); // Apple console.log(fruits[fruits.length - 1]); // Mango</pre>	
Changing Array Elements:	
<pre>fruits[1] = "Orange"; // changes Banana to Orange</pre>	

Common Array Properties:

Property	Description	Example
<code>.length</code>	Number of elements	<code>fruits.length</code> → 3

Example code:	Output:
<pre>let fruits = ["Apple", "Banana", "Mango"]; let length = fruits.length; console.log("Number of fruits:", length);</pre>	Number of fruits: 3

Common Array Methods:

Method	Description	Example
<code>push(item)</code>	Adds to end	<code>arr.push("Grapes")</code>
<code>pop()</code>	Removes last item	<code>arr.pop()</code>
<code>unshift(item)</code>	Adds to start	<code>arr.unshift("Lemon")</code>
<code>shift()</code>	Removes first item	<code>arr.shift()</code>
<code>concat(arr2)</code>	Joins arrays	<code>arr.concat(arr2)</code>
<code>join(separator)</code>	Array → string	<code>arr.join(", ")</code>
<code>slice(start, end)</code>	Copies part of array	<code>arr.slice(1, 3)</code>
<code>splice(start, deleteCount, ...items)</code>	Adds/removes items	<code>arr.splice(1, 1, "Kiwi")</code>
<code>indexOf(value)</code>	First index of value	<code>arr.indexOf("Apple")</code>
<code>lastIndexOf(value)</code>	Last index of value	<code>arr.lastIndexOf("Apple")</code>
<code>includes(value)</code>	Checks if exists	<code>arr.includes("Banana")</code>
<code>reverse()</code>	Reverses array	<code>arr.reverse()</code>
<code>sort()</code>	Sorts alphabetically	<code>arr.sort()</code>
<code>toString()</code>	Array → string	<code>arr.toString()</code>
<code>flat(depth)</code>	Flattens nested arrays	<code>[1, [2, 3]].flat()</code>

Multidimensional Arrays

Example code:	Output:
<pre>let matrix = [[1, 2, 3], [4, 5, 6]]; console.log(matrix[1][2]);</pre>	6

Looping Through Arrays:

Example code:	Output:
<pre>let fruits = ["Apple", "Banana", "Mango"]; // for loop for (let i = 0; i < fruits.length; i++) { console.log(fruits[i]); }</pre>	Apple Banana Mango

Practice Problem

🔗 **Problem1:** For a given array with marks of students → [85, 97, 44, 37, 76, 60] Find the average marks of the entire class.

Code:	Output:
<pre>let marks = [85, 97, 44, 37, 76, 60]; let sum = 0; for (let i = 0; i < marks.length; i++) { sum += marks[i]; } let average = sum / marks.length; console.log("Average Marks:", average);</pre>	Average Marks: 66.5

🔗 **Problem2:** For a given array with prices of 5 items → [250, 645, 300, 900, 50] All items have an offer of 10% OFF on them. Change the array to store final price after applying offer.

Code:	Output:
<pre>let prices = [250, 645, 300, 900, 50]; for (let i = 0; i < prices.length; i++) { let discount = prices[i] * 0.10; prices[i] = prices[i] - discount; } console.log("Final Prices after 10% OFF:", prices);</pre>	Final Prices after 10% OFF: (5) [225, 580.5, 270, 810, 45]

- ⌘ **Problem3:** Create an array to store companies → “Bloomberg”, “Microsoft”, “Uber”, “Google”, “IBM”, “Netflix”
- Remove the first company from the array
 - Remove Uber & Add Ola in its place
 - Add Amazon at the end

Code:	Output:
<pre>let companies = ["Bloomberg", "Microsoft", "Uber", "Google", "IBM", "Netflix"]; // a. Remove the first company companies.shift(); // b. Remove Uber & Add Ola in its place let uberIndex = companies.indexOf("Uber"); if (uberIndex !== -1) { companies.splice(uberIndex, 1, "Ola"); } // c. Add Amazon at the end companies.push("Amazon"); console.log(companies);</pre>	<pre>(6) ['Microsoft', 'Ola', 'Google', 'IBM', 'Netflix', 'Amazon']</pre>

FUNCTIONS

Functions

➔ A **function** is a block of reusable code that performs a specific task.

Define a Function:	Call a Function:
<pre>function function_name(){ console.log("Hello World!"); }</pre>	<pre>function_name();</pre>
Example code:	Output:
<pre>function greet(name) { console.log("Hello " + name); } greet("Mohsin");</pre>	Hello, Mohsin
Notes: Use function to define a function.	

Function Expression

⇒ A function can be stored in a variable.

Example code:	Output:
<pre>const greet = function(name) { console.log("Hello " + name); }; greet("Alif");</pre>	Hello Alif
Notes: Functions help make code modular and reusable .	

Arrow Functions

Example code:	Output:
<pre>const greet = (name) => { console.log("Hello " + name); }; greet("Jam");</pre>	Hello Jam

Function with Return Value

Example code:	Output:
<pre>function add(a, b) { return a + b; } let sum = add(5, 3); console.log(sum); // 8</pre>	8

Default Parameters

Example code:	Output:
<pre>function greet(name = "Guest") { console.log("Hello " + name); } greet(); // Hello Guest greet("Nihan"); // Hello Nihan</pre>	Hello Guest Hello Nihan

Rest Parameters

⇒ Allows a function to accept **any number of arguments**.

Example code:	Output:
<pre>function sum(...numbers) { let total = 0; for (let num of numbers) { total += num; } return total; } console.log(sum(1, 2, 3, 4)); // 10</pre>	10

Anonymous Functions

⇒ Functions without a name, usually used as arguments.

Example code:	Output:
<pre>setTimeout(function() { console.log("Hello after 2 seconds"); }, 2000);</pre>	Hello after 2 seconds

IIFE (Immediately Invoked Function Expression)

⇒ A function that runs **immediately** after it's defined.

Example code:	Output:
<pre>(function() { console.log("IIFE runs immediately"); })();</pre>	IIFE runs immediately

Scope in Functions

⇒ Variables inside a function are **local**.

⇒ Variables outside are **global**.

Example code:	Output:
<pre>let globalVar = 10; function test() { let localVar = 5; console.log(globalVar); // 10 console.log(localVar); // 5 } test(); console.log(localVar); // ❌ Error</pre>	10 5

Practice Problem

🔗 **Problem1:** Create a function using the “function” keyword that takes a String as an argument & returns the number of vowels in the string.

Code:	Output:
<pre>function countVowels(str) { let count = 0; let vowels = "aeiouAEIOU"; for (let i = 0; i < str.length; i++) { if (vowels.includes(str[i])) { count++; } } return count; } console.log(countVowels("Hello World"));</pre>	3

🔗 **Problem2:** Create an arrow function to perform the same task

Code:	Output:
<pre>const countVowels = (str) => { let count = 0; let vowels = "aeiouAEIOU"; for (let i = 0; i < str.length; i++) { if (vowels.includes(str[i])) { count++; } } return count; }; console.log(countVowels("Hello World"));</pre>	3

✂ **Problem3:** Create an arrow function to perform the same task

Code:	Output:
<pre>let numbers = [2, 4, 6, 8, 10]; numbers.forEach(function(num) { console.log(num * num); });</pre>	<pre>4 16 36 64 100</pre>

Problem4: We are given array of marks of students. Filter out of the marks of students that scored 90+.

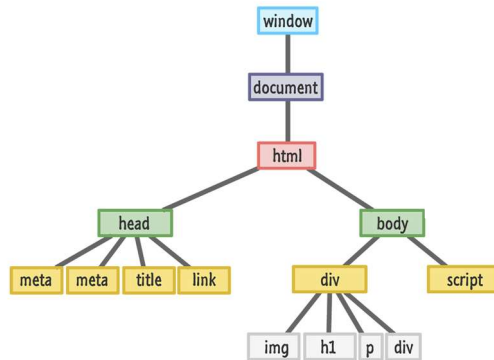
- Take a number n as input from user. Create an array of numbers from 1 to n.
- Use the reduce method to calculate sum of all numbers in the array.
- Use the reduce method to calculate product of all numbers in the array.

Code: Filter students with marks 90+	Output:
<pre>var marks = [85, 97, 44, 93, 76, 99, 91]; var highScorers = marks.filter(function(mark) { return mark > 90; }); console.log("Marks 90+ :", highScorers);</pre>	<pre>Marks 90+ : (4) [97, 93, 99, 91]</pre>
Code: Create array from 1 to n	Output:
<pre>var n = parseInt(prompt("Enter a number:")); var arr = []; for (var i = 1; i <= n; i++) { arr.push(i); } console.log("Array from 1 to n:", arr);</pre>	<pre>Array from 1 to n: (5) [1, 2, 3, 4, 5]</pre>
Code: Calculate product using reduce	Output:
<pre>var arr = [1,2,3,4,5]; var product = arr.reduce(function(accumulator, currentValue) { return accumulator * currentValue; }, 1); console.log("Product of numbers:", product);</pre>	<pre>Product of numbers: 120</pre>

DOM MANIPULATION

DOM Manipulation

- **DOM** (Document Object Model) represents the **HTML structure** of a webpage.
- JavaScript can **access, change, add, or remove HTML elements** using the DOM.



Accessing Elements

Method	Description	Example
<code>getElementById()</code>	Select element by id	<code>document.getElementById("myDiv")</code>
<code>getElementsByClassName()</code>	Select elements by class	<code>document.getElementsByClassName("myClass")</code>
<code>getElementsByTagName()</code>	Select elements by tag	<code>document.getElementsByTagName("p")</code>
<code>querySelector()</code>	Select first element matching CSS selector	<code>document.querySelector(".myClass")</code>
<code>querySelectorAll()</code>	Select all elements matching CSS selector	<code>document.querySelectorAll("p")</code>

Changing Content

Property / Method	Description	Example
<code>innerHTML</code>	Change HTML content	<code>element.innerHTML = "<div>Hello</div>"</code>
<code>textContent</code>	Change text content only	<code>element.textContent = "Hello"</code>
<code>innerText</code>	Change visible text	<code>element.innerText = "Hello"</code>

Changing Styles using DOM

Example code:	Output:	
<pre>let box = document.getElementById("Box"); box.style.backgroundColor = "red"; box.style.width = "200px"; box.style.border = "2px solid black";</pre>	Before JS	After JS
		

Changing Attributes

Method	Description	Example
<code>getAttribute()</code>	Get attribute value	<code>element.getAttribute("src")</code>
<code>setAttribute()</code>	Set attribute value	<code>element.setAttribute("src", "image.jpg")</code>
<code>removeAttribute()</code>	Remove an attribute	<code>element.removeAttribute("disabled")</code>

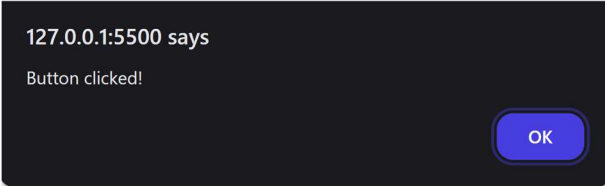
Example code:	Output:
<pre><div id="Box" title="Original Title">Box</div> <script> // Select the element let box = document.getElementById("Box"); // getAttribute() console.log("Title before:", box.getAttribute("title")); // setAttribute() box.setAttribute("title", "Updated Title"); console.log("Title After:", box.getAttribute("title")); // removeAttribute() box.removeAttribute("title"); console.log("Title After remove:", box.getAttribute("title")); </script></pre>	<pre>Title before: Original Title Title After: Updated Title Title After remove: null</pre>

Adding / Removing Elements

Example code:	Output:
<pre><div id="Box" title="Original Title">Box</div> <div id="Box2"></div> <script> // Create a new element let p = document.createElement("p"); p.textContent = "Hello World"; let div2 = document.getElementById("Box2"); // Append to body or another element div2.appendChild(p); // Remove an element let div = document.getElementById("Box"); div.remove(); </script></pre>	<pre><body> <div id="Box2"> <p>Hello World</p> </div> <script>...</script> </body></pre>

Event Handling

⇒ Respond to **user actions** like **clicks**, **mouse movements**, **keyboard input**.

Example code:	Output:
<pre><button id="myBtn">Click me</button> <script> let button = document.getElementById("myBtn"); button.addEventListener("click", function () { alert("Button clicked!"); }); </script></pre>	

➤ **Common Events:**

Event Name	Trigger Description
<code>click</code>	Mouse click
<code>mouseover</code>	Mouse moves over the element
<code>mouseout</code>	Mouse leaves the element
<code>keydown</code>	A key is pressed on the keyboard
<code>keyup</code>	A key is released on the keyboard
<code>submit</code>	A form is submitted

Traversing the DOM

Property	Description	Example
<code>parentNode</code>	Access parent element	<code>element.parentNode</code>
<code>children</code>	Access child elements	<code>element.children</code>
<code>firstChild</code>	First child node	<code>element.firstChild</code>
<code>lastChild</code>	Last child node	<code>element.lastChild</code>
<code>nextSibling</code>	Next sibling node	<code>element.nextSibling</code>
<code>previousSibling</code>	Previous sibling node	<code>element.previousSibling</code>

Example code: Traversing the DOM

```
<div id="parent">
  <p id="first">First paragraph</p>
  <p id="second">Second paragraph</p>
  <p id="third">Third paragraph</p>
</div>

<script>
  let secondPara = document.getElementById("second");

  // 1. parentNode
  console.log("Parent Node of second:", secondPara.parentNode);

  // 2. children
  console.log("Children of parent div:", secondPara.parentNode.children);

  // 3. firstChild
  console.log("First Child of parent div:", secondPara.parentNode.firstChild);

  // 4. lastChild
  console.log("Last Child of parent div:", secondPara.parentNode.lastChild);

  // 5. nextSibling
  console.log("Next Sibling of second paragraph:", secondPara.nextSibling);

  // 6. previousSibling
  console.log("Previous Sibling of second paragraph:", secondPara.previousSibling);
</script>
```

Output:

```
parentNode: <div id="parent">...</div>
children: HTMLCollection(3) [<p id="first">...</p>, <p id="second">...</p>, <p id="third">...</p>]
firstChild: <p id="first">First paragraph</p>
lastChild: <p id="third">Third paragraph</p>
nextSibling: <p id="third">Third paragraph</p>
previousSibling: <p id="first">First paragraph</p>
```

Example code: Event Handling

```
<button id="btnClick">Click Me</button><br><br>

<div id="hoverBox">Hover Here</div><br>

<input id="keyInput" type="text" placeholder="Type something..."><br><br>

<form id="myForm">
  <input type="text" placeholder="Your name">
  <button type="submit">Submit</button>
</form>

<script>
  // Elements
  const btnClick = document.getElementById("btnClick");
  const hoverBox = document.getElementById("hoverBox");
  const keyInput = document.getElementById("keyInput");
  const myForm = document.getElementById("myForm");

  // Actions
  const handleClick = () => {
    console.log("Button clicked!");
  };

  const handleMouseOver = () => {
    console.log("Mouse over the box!");
    hoverBox.style.backgroundColor = "lightgreen";
  };

  const handleMouseOut = () => {
    console.log("Mouse out of the box!");
    hoverBox.style.backgroundColor = "lightblue";
  };

  const handleKeyDown = (event) => {
    console.log("Key down:", event.key);
  };

  const handleKeyUp = (event) => {
    console.log("Key up:", event.key);
  };
</script>
```

```
const handleSubmit = (event) => {  
  event.preventDefault();  
  console.log("Form submitted!");  
};  
  
// Event Listeners  
btnClick.addEventListener("click", handleClick);  
hoverBox.addEventListener("mouseover", handleMouseOver);  
hoverBox.addEventListener("mouseout", handleMouseOut);  
keyInput.addEventListener("keydown", handleKeyDown);  
keyInput.addEventListener("keyup", handleKeyUp);  
myForm.addEventListener("submit", handleSubmit);  
</script>
```

Output:

Button clicked!

Mouse over the box!

Mouse out of the box!

Key down: b

Key up: b

Form submitted!

Example code: Changing Content

```
<div id="demo1">Original <b>HTML</b> content</div>
<div id="demo2">Original <b>HTML</b> content</div>
<div id="demo3">Original <b>HTML</b> content</div>

<button id="btnHTML">Change innerHTML</button>
<button id="btnTextContent">Change textContent</button>
<button id="btnInnerText">Change innerText</button>

<script>
  const demo1 = document.getElementById("demo1");
  const demo2 = document.getElementById("demo2");
  const demo3 = document.getElementById("demo3");

  const btnHTML = document.getElementById("btnHTML");
  const btnTextContent = document.getElementById("btnTextContent");
  const btnInnerText = document.getElementById("btnInnerText");

  // innerHTML changes HTML structure
  btnHTML.onclick = () => {
    demo1.innerHTML = "<i>Hello</i> <b>World</b>";
  };

  // textContent changes only text (HTML tags shown as plain text)
  btnTextContent.onclick = () => {
    demo2.textContent = "<i>Hello</i> <b>World</b>";
  };

  // innerText changes visible text (similar to textContent but respects CSS styles)
  btnInnerText.onclick = () => {
    demo3.innerText = "<i>Hello</i> <b>World</b>";
  };
</script>
```

Output: Before

Original **HTML** content
Original **HTML** content
Original **HTML** content

Change innerHTML

Change textContent

Change innerText

Output: After

Hello **World**

<i>Hello</i> World

<i>Hello</i> World

Change innerHTML

Change textContent

Change innerText

Example code: Accessing Elements

```
<div id="myDiv">This is a div with id "myDiv"</div>

<p class="myClass">Paragraph 1 with class "myClass"</p>
<p class="myClass">Paragraph 2 with class "myClass"</p>
<p>Paragraph 3 with no class</p>

<button id="btn">Select Elements</button>

<script>
  const btn = document.getElementById("btn");

  btn.onclick = () => {
    // 1. getElementById
    const divById = document.getElementById("myDiv");
    console.log("getElementById:", divById);

    // 2. getElementsByName
    const elemsByClass = document.getElementsByClassName("myClass");
    console.log("getElementsByName:", elemsByClass);

    // 3. getElementsByTagName
    const elemsByTag = document.getElementsByTagName("p");
    console.log("getElementsByTagName:", elemsByTag);

    // 4. querySelector (first match)
    const firstByQuery = document.querySelector(".myClass");
    console.log("querySelectorClass:", firstByQuery);

    // 5. querySelector by id
    const idByQuery = document.querySelector("#myDiv");
    console.log("querySelectorId:", idByQuery);

    // 6. querySelectorAll (all matches)
    const allByQuery = document.querySelectorAll("p");
    console.log("querySelectorAll:", allByQuery);
  };
</script>
```